WEB APPLICATION FOR SECURE TRANSMISSION OF DATA

A Project Report Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

M. RAMA SUBBARAO	**22221A0570**
M. GANGADHAR RAO	**22221A0569**
M. RAKESH SURYA GANGADHAR	**22221A0572**
B. SIVA RAMA KRISHNA	**22221A0517**
B. ROHITH SAI RAMAREDDY	**22221A0513**

Under the Esteemed Guidance of

Mrs. B. GANGA BHAVANI

M. Tech, (Ph. D)

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Permanently Affiliated to J. N.T.U.K, Kakinada)

(Accredited by N.B.A & NAAC with 'A' Grade)

ODALAREVU – 533210

2022-2026

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE
(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Permanently Affiliated to J. N.T.U.K, Kakinada)

(Accredited by N.B.A & NAAC with 'A' Grade)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work entitled "**WEB APPLICATION FOR SECURE TRANSMISSION OF DATA**" is being submitted for the partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering, at BVC Engineering College, Odalarevu**, is a bonafide work done by **M. RAMA SUBBARAO (22221A0570), M. GANGADHAR RAO (22221A0569), M. RAKESH SURYA GANGADHAR (22221A0572), B. SIVA RAMA KRISHNA (22221A0517), B. ROHITH SAI RAMAREDDY (22221A0513).** under my guidance during the academic year 2025-2026 and it has been found suitable for acceptance according to the requirement of the University.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree.

Project Guide
Mrs. B. GANGA BHAVANI
M. Tech, (Ph. D)
Associate Professor

Head of the Department
Dr. MEERASHARIF SHEIK
M. Tech, Ph. D
Professor& HOD

External Examiner

ACKNOWLEDGEMENT

We profoundly grateful to express our deep sense of gratitude and respect towards our guide Mrs. B. GANGA BHAVANI, M.Tech, (Ph.D), Associate Professor of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work. His constant encouragement and support has been the cause of our success, in completing this thesis in the college.

We also grateful to the Project Co-Ordinator Mr.M.V.K.SUBASH, M.Tech, (Ph.D), Assistant Professor of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for her excellent guidance right from selection of the project and her valuable suggestions throughout the project work.

We really thankful to Dr. MEERASHARIF SHEIK, M.Tech, Ph.D, Professor and Head of the Department of Computer Science and Engineering for providing the laboratory facilities to the fullest extent as and when required and also for giving us the opportunity to carry out the project work in the college.

We also thankful to Dr. MAHESWAR DUTTA, M.E, Ph.D, Principal, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his support during and till the completion of the project. And also thankful to our MANAGEMENT, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for their support during and till the completion of the project.

We are thankful to all the TEACHING AND NON-TEACHING STAFF of Computer Science and Engineering Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, and FRIENDS for their direct and indirect help provided to us in completing the project.

Last but not least, we own all our success to our FAMILY MEMBERS, CLASSMATES AND TEACHERS from our child hood, whose vision, love and inspiration made us to reach out for these glories.

PROJECT ASSOCIATES

M. RAMA SUBBARAO	**22221A0570**
M. GANGADHAR RAO	**22221A0569**
M. RAKESH SURYA GANGADHAR	**22221A0572**
B. SIVA RAMA KRISHNA	**22221A0517**
B. ROHITH SAI RAMAREDDY	**22221A0513**

ABSTRACT

The Web Application for Secure Transmission of Data is designed to provide a robust, reliable, and secure platform for exchanging sensitive information over the internet. With the rapid growth of digital communication and increasing cyber threats, ensuring secure data transmission has become a critical requirement for organizations and individuals alike. This application addresses these concerns by implementing strong cryptographic techniques and secure communication mechanisms. Developed using the Python Flask framework, the system ensures confidentiality, integrity, and authenticity of data throughout the transmission process. Users can register, log in, and authenticate securely before accessing the system. A clean, responsive, and user-friendly interface built using HTML and CSS allows users to easily share and manage sensitive information without technical complexity.

To protect data from unauthorized access, the application integrates the Advanced Encryption Standard (AES) for efficient symmetric encryption and the Rivest–Shamir–Adleman (RSA) algorithm for secure key exchange and authentication between communicating parties. This hybrid encryption approach ensures that data remains encrypted during transmission and is protected against common security threats such as eavesdropping, data tampering, and man-in-the-middle attacks. The backend of the application is powered by MySQL, with Flask's Object Relational Mapping (ORM) facilitating secure, efficient, and organized database operations. The system architecture follows a modular design, allowing easy maintenance, future enhancements, and scalability. Security best practices such as encrypted storage, access control, and secure session management are also incorporated.

LIST OF CONTENTS

CONTENTS **PAGE NO**

CHAPTER – 1 INTRODUCTION…….. 1 - 3

1.1 Overview of Secure Data Transmission……..... 1

1.2 Need for Data Security……... 2

1.3 Objectives of the Project…… 3

1.4 Scope of the Project……... 3

CHAPTER – 2 LITERATURE SURVEY…… 4 - 8

2.1 Existing Research on Secure Web Applications…… 4

2.2 Cryptographic Techniques in Data Transmission…….. 6

CHAPTER – 3 SYSTEM ANALYSIS……. 9 - 11

3.1 Existing System……. 9

3.1.1 Drawbacks of Existing System……. 10

3.2 Proposed System…… 10

3.2.1 Advantages of Proposed System……... 11

CHAPTER – 4 REQUIREMENT ANALYSIS…… 12

4.1 Hardware Requirements……. 12

4.2 Software Requirements……... 12

CHAPTER – 5 SOFTWARE DESIGN……. 13 - 15

5.1 System Architecture…… 13

5.2 UML Diagrams……... 13

5.2.1 Use Case Diagram……. 14

5.2.2 Class Diagram…… 14

5.2.3 Sequence Diagram……. 15

CONTENTS **PAGE NO**

CHAPTER – 6 SYSTEM ENVIRONMENT & TECHNOLOGIES……. 16 - 37

6.1 Python Programming Language……. 16

6.2 Flask Web Framework…… 28

6.3 HTML and CSS……... 29

6.4 MySQL Database…… 30

6.5 Cryptography Overview……... 31

6.5.1 AES Algorithm……... 33

6.5.2 RSA Algorithm……... 36

CHAPTER – 7 MODULES USED IN PROJECT…… 38 - 40

7.1 User Registration and Authentication Module…… 38

7.2 Encryption and Decryption Module……. 38

7.3 Secure Data Transmission Module……... 39

7.4 Database Management Module…… 39

7.5 Installation of Python……... 40

CHAPTER – 8 TESTING…… 41 - 44

8.1 System Testing……. 41

8.2 Types of Testing……... 41

8.2.1 Unit Testing……. 41

8.2.2 Integration Testing……... 41

8.2.3 Functional Testing……... 42

8.2.4 White Box Testing……... 42

8.2.5 Black Box Testing……... 43

CHAPTER – 9 IMPLEMENTATION…… 45 - 57

9.1 Methodology……. 45

9.2 Source Code Description……... 48

CHAPTER – 10 RESULTS AND OUTPUT SCREENS……... 58 - 62

CHAPTER – 11 CONCLUSION AND FUTURE ENHANCEMENTS……... 63

CHAPTER – 12 REFERENCES……. 64 - 65

LIST OF FIGURES

FIGURES **PAGE NO**

Fig 5.1 System Architecture of Secure Data Transmission Application……... 13

Fig 5.2 Class Diagram……... 14

Fig 5.3 Use Case Diagram……. 14

Fig 5.4 Sequence Diagram…… 15

Fig 6.1 Open Official Python Website to Download Python…… 21

Fig 6.2 Click on Download Python Option……... 21

Fig 6.3 Select Required Python Version……... 22

Fig 6.4 Python Version Selection Screen…….. 22

Fig 6.5 Click on Install Python Setup…… 23

Fig 6.6 Click on Install Now……. 23

Fig 6.7 Python Installation Successfully Completed……. 24

Fig 6.8 Open Command Prompt…… 24

Fig 6.9 Verify Python Installation Using Python –V Command……... 25

Fig 6.10 Launch Flask Application Using Command Line……... 26

Fig 10.1 Login Interface…… 55

Fig 10.2 Algorithm Selection…… 55

Fig 10.3 AES Operation Menu……. 56

Fig 10.4 AES Encryption Page……. 56

Fig 10.5 Encryption Result…… 57

Fig 10.6 AES Decryption Page……. 57

Fig 10.7 RSA Operation Dashboard……. 58

Fig 10.8 RSA Encryption Screen…….. 58

Fig 10.9 RSA Decryption Screen…….. 59

Fig 10.10 Project Folder Structure…… 59

CHAPTER - 1

INTRODUCTION

1.1 OVERVIEW OF SECURE DATA TRANSMISSION

Secure transmission of data is a critical requirement in today's digital era, where vast amounts of sensitive information are exchanged over the internet every second. With the rapid growth of web-based applications, cloud services, and online communication platforms, ensuring the confidentiality, integrity, and authenticity of data has become a major concern for individuals, organizations, and governments. Cyber threats such as data breaches, eavesdropping, man-in-the-middle attacks, and unauthorized access pose significant risks to digital communication systems.

To address these challenges, modern cryptographic techniques play a vital role in protecting data during transmission. Encryption algorithms such as Advanced Encryption Standard (AES) and Rivest–Shamir–Adleman (RSA) have proven to be highly effective in securing information by transforming readable data into an unreadable format for unauthorized users. AES provides fast and efficient symmetric encryption for large data, while RSA enables secure key exchange and authentication using asymmetric cryptography. The combination of these techniques ensures a robust and reliable security mechanism for data transmission.

This project focuses on the development of a web application for secure transmission of data using a hybrid encryption approach. The application is built using the Python Flask framework, which provides a lightweight, flexible, and scalable environment for web development. Users can securely register, authenticate, and exchange sensitive data through a user-friendly interface developed using HTML and CSS. All transmitted data is encrypted before being sent over the network and decrypted only by authorized recipients, ensuring end-to-end security.

By integrating AES for data encryption and RSA for secure key management, the system enhances protection against common cyber threats while maintaining high performance and usability. The proposed application aims to reduce reliance on insecure communication methods and provide a practical, scalable solution for secure digital communication. This web- based secure data transmission system is suitable for applications in business communication, academic data sharing, healthcare systems, and other domains where data security is of utmost importance.

1.2 NEED FOR DATA SECURITY

In today's digital era, the rapid growth of web applications and online communication has led to an exponential increase in the exchange of sensitive information over the internet. Data such as personal details, financial records, login credentials, and confidential organizational information are constantly transmitted across networks. This makes data security a critical requirement for any web-based system. Without proper security mechanisms, data transmitted over the internet is vulnerable to threats such as unauthorized access, data interception, modification, and cyberattacks.

The need for data security arises from the growing number of cyber threats including hacking, phishing, man-in-the-middle attacks, and data breaches. Attackers can exploit vulnerabilities in unsecured web applications to steal or manipulate data, leading to financial loss, identity theft, and damage to user trust. Ensuring secure data transmission helps protect information from being accessed or altered by unauthorized entities during communication between users and servers.

1.3 OBJECTIVES OF THE PROJECTS

The main objective of the **Web Application for Secure Transmission of Data** is to design and develop a secure and reliable platform that enables safe exchange of information over the internet. The project focuses on implementing effective security mechanisms to protect data from unauthorized access and cyber threats.

The specific objectives of the project are:

1. **To ensure secure data transmission** by implementing encryption techniques that protect data during transfer between users and servers.
2. **To maintain data confidentiality** by allowing access only to authorized users through secure authentication and authorization mechanisms.
3. **To preserve data integrity** by preventing unauthorized modification of data during transmission and storage.
4. **To provide a user-friendly web interface** that allows users to send and receive data securely with ease.
5. **To prevent common web security threats** such as data interception, unauthorized access, and man-in-the-middle attacks.
6. **To implement secure session management** to protect user sessions from hijacking and unauthorized usage.

7. **To reduce dependency on manual security practices** by automating secure data handling within the web application.
8. **To develop a scalable and efficient system** that can be extended to support advanced security features in the future.

1.4 SCOPE OF THE PROJECT

The scope of the **Web Application for Secure Transmission of Data** is to provide a secure and efficient platform for transmitting sensitive information over the internet. The project focuses on implementing essential security mechanisms such as encryption, authentication, and secure communication protocols to protect data from unauthorized access and cyber threats during transmission.

This project covers the design and development of a web-based system that enables users to securely upload, send, and access data through a controlled environment. It includes user authentication, secure data storage, encrypted data transmission, and session management to ensure confidentiality and integrity of information. The application is intended for use in environments where secure communication is critical, such as academic institutions, organizations, and small-scale enterprises.

The system is designed to handle structured and unstructured data while ensuring protection against common security vulnerabilities like data interception, unauthorized access, and data tampering. The project scope also includes basic administrative controls for managing users and monitoring system activity to enhance overall security.

However, the scope of the project is limited to a web-based implementation and does not include advanced features such as blockchain-based security, intrusion detection systems, or large-scale distributed architectures. Future enhancements may expand the system to include advanced encryption techniques, multi-factor authentication, mobile application support, and cloud-based secure communication.

Overall, this project aims to demonstrate a practical and effective approach to secure data transmission using web technologies, providing a foundation that can be enhanced and scaled for real-world security applications.

CHAPTER - 2
LITERATURE SURVEY

2.1 EXISTING RESEARCH ON SECURE WEB APPLICATIONS

1. Secure Communication Protocols

Research has extensively focused on protocols that ensure secure communication between clients and servers. The **Transport Layer Security (TLS)** protocol, including versions TLS 1.2 and TLS 1.3, are widely studied as the foundations for secure data transmission in web applications. These protocols provide encryption and authentication, preventing eavesdropping and tampering of data over networks.

- *Rescorla (2018)* describes how TLS 1.3 improves performance and security compared to earlier versions by reducing handshake latency and strengthening cryptographic algorithms.
- Studies have shown that TLS adoption significantly reduces vulnerability to attacks like man-in-the-middle (MITM) and session hijacking.

2. Encryption Techniques

Encryption is a core aspect of secure web applications. Research in cryptography highlights both symmetric and asymmetric encryption methods used to secure user data.

- **Advanced Encryption Standard (AES)** is widely used for encrypting data due to its strong security and efficiency.
- Public-key cryptography techniques, such as RSA and Elliptic Curve Cryptography (ECC), are used to securely exchange keys over insecure channels.

Academic literature emphasizes that combining these techniques provides a robust framework for protecting data both in transit and at rest.

3. Authentication and Access Control

Authentication research investigates secure and user-friendly methods to verify identities and control access.

- Multi-factor authentication (MFA) has been shown to significantly reduce unauthorized access compared to simple password-only systems.
- OAuth and JWT (JSON Web Tokens) are widely adopted standards for managing user sessions securely in web applications.

Studies suggest that enhanced authentication methods improve both security and user trust in web services.

4. Session Management and Secure Cookies

Session management is critical in maintaining secure communication once a user is authenticated.

- Research shows that secure cookie attributes (such as HttpOnly and Secure flags) prevent session hijacking and cross-site scripting (XSS) attacks.
- Techniques like session token rotation and timeout mechanisms are recommended to minimize attack windows.

5. Secure Coding and Vulnerability Prevention

A significant body of research focuses on secure development practices to prevent vulnerabilities.

- The **OWASP Top 10** outlines common security risks for web applications, including SQL injection, cross-site scripting (XSS), and broken authentication.
- Research promotes secure coding standards and tools that identify vulnerabilities early in development, reducing the risk of security flaws in deployed systems.

6. Security in Web Application Frameworks

Modern web frameworks (such as Django, Express, and ASP.NET) include built-in security features.

- Research examines how frameworks integrate protections against common attacks and provide secure defaults.
- Framework-level security reduces the burden on developers and encourages best practices.

7. Real-World Case Studies

Several case studies in literature demonstrate the impact of secure web applications:

- Implementation of HTTPS across major websites has led to decreased data breaches and improved user privacy.
- Studies on secure e-commerce platforms show that encryption and dynamic session management are essential for building customer trust.

2.2 CRYPTOGRAPHIC TECHNIQUES IN DATA TRANSMISSION

Cryptographic techniques play a vital role in ensuring secure data transmission in web applications. These techniques protect sensitive information from unauthorized access, interception, and modification while data is being transmitted over the network. By using encryption, authentication, and integrity verification methods, cryptography ensures safe communication between users and servers.

1. Encryption Techniques

Encryption converts readable data (plaintext) into an unreadable format (ciphertext) to prevent unauthorized access.

a) Symmetric Encryption

Symmetric encryption uses a single secret key for both encryption and decryption.

- **Advanced Encryption Standard (AES)** is widely used for secure data transmission due to its high speed and strong security.
- It is suitable for encrypting large amounts of data efficiently.
- Commonly used in HTTPS and secure file transfer systems.

Advantages:

- Fast and efficient
- Suitable for real-time data transmission

Limitations:

- Secure key exchange is required

b) Asymmetric Encryption

Asymmetric encryption uses a pair of keys: a public key and a private key.

- **RSA** and **Elliptic Curve Cryptography (ECC)** are commonly used algorithms.
- Public keys encrypt the data, while private keys decrypt it.
- Primarily used for secure key exchange and authentication.

Advantages:

- Secure key distribution
- Strong authentication

Limitations:

- Slower than symmetric encryption

2. Key Exchange Mechanisms

Secure key exchange is essential to establish a shared secret key between communicating parties.

- **Diffie-Hellman (DH)** and **Elliptic Curve Diffie-Hellman (ECDH)** are widely used for secure key exchange.
- These methods allow two parties to securely generate a shared secret over an insecure network.
- Used extensively in TLS/SSL protocols.

3. Hashing Techniques

Hashing ensures data integrity by generating a fixed-size hash value from input data.

- Common algorithms include **SHA-256** and **SHA-3**.
- Even a small change in data produces a completely different hash.
- Used for password storage and data integrity verification.

Applications:

- Password protection
- Message integrity checking

4. Digital Signatures

Digital signatures provide authentication and non-repudiation.

- Created using asymmetric cryptography.
- The sender signs data with a private key, and the receiver verifies it using the public key.
- Ensures the sender's identity and confirms that data has not been altered.

5. Secure Communication Protocols

Cryptographic techniques are implemented through secure protocols.

- **SSL/TLS** encrypts data between web browsers and servers.
- Provides encryption, authentication, and integrity.
- Ensures secure HTTPS communication in web applications.

6. Authentication and Authorization

Cryptography supports secure user authentication.

- Passwords are hashed and salted before storage.
- Tokens (such as JWT) are cryptographically signed to prevent tampering.
- Enhances access control and session security.

Role of Cryptography in the Project

In the **Web Application for Secure Transmission of Data**, cryptographic techniques ensure:

- Confidentiality of transmitted data
- Integrity of information
- Secure user authentication
- Protection against cyber threats such as man-in-the-middle attacks

CHAPTER - 3
SYSTEMANALYSIS

3.1 Existing System

Existing systems for secure data transmission primarily rely on conventional web-based communication mechanisms combined with basic security protocols. Many web applications transmit sensi–tive information such as personal data, credentials, and confidential documents over the internet, often without sufficient encryption or secure authentication mechanisms. Although standard protocols like HTTPS provide a baseline level of security, many legacy or poorly implemented systems remain vulnerable to cyber threats such as eavesdropping, data tampering, unauthorized access, and man-in-the-middle attacks.

Traditional secure communication systems use password-based authentication and centralized storage, where data is stored either in plaintext or with weak encryption techniques. In several cases, symmetric encryption methods are used without proper key management, making systems susceptible to key leakage and replay attacks. These systems lack end-to-end encryption, meaning data may be exposed at intermediate stages during transmission or storage. As a result, attackers can exploit vulnerabilities to intercept or manipulate sensitive data.

Many existing web-based security systems depend on standalone encryption solutions or third-party tools that are difficult to integrate and manage. They often require complex configurations and lack user-friendly interfaces, making them unsuitable for non-technical users. Furthermore, scalability and flexibility remain major challenges, as most systems are not designed to adapt to increasing user demands or evolving security requirements.

Here's a well-structured write-up for the Existing Secure Data Transmission Approaches

1. Traditional Encryption-Based Systems

Earlier systems relied on basic encryption algorithms such as DES or simple symmetric key encryption. While these approaches provided some level of data protection, they suffered from weak key sizes, inefficient performance, and poor resistance to modern cryptographic attacks.

2. Password-Based Authentication Systems

Many applications implemented simple username and password authentication without

secure hashing or salting techniques. Such systems were vulnerable to brute-force attacks, credential theft, and unauthorized access due to weak password management practices.

3. Secure Socket Layer (SSL/TLS) Based Communication

SSL/TLS protocols improved data security during transmission by encrypting communication channels. However, improper certificate management, misconfigurations, and lack of application-level encryption still left systems exposed to advanced attacks.

4. Symmetric Encryption Systems

Symmetric algorithms like AES are widely used due to their speed and efficiency. However, existing systems often struggle with secure key distribution and management, especially in multi-user environments.

5. Asymmetric Encryption Systems

Asymmetric algorithms such as RSA provide secure key exchange and authentication. Nevertheless, their higher computational cost makes them unsuitable for encrypting large volumes of data directly.

3.1.1 Drawbacks

The existing systems for secure data transmission suffer from several limitations that impact overall security and performance. One major drawback is improper key management, where encryption keys are stored or transmitted insecurely, increasing the risk of data compromise. Additionally, many systems do not implement hybrid encryption techniques, resulting in either high computational overhead or weak security.

Another significant limitation is the lack of robust authentication and access control mechanisms. Systems relying solely on passwords are vulnerable to credential-based attacks, while inadequate session management can lead to session hijacking. Performance degradation is also observed in systems using only asymmetric encryption for data protection, making them inefficient for real-time applications.

Furthermore, existing systems often fail to provide scalability and flexibility. As user numbers increase, managing secure communication becomes more complex, and system performance may degrade. The absence of modular architecture also makes it difficult to update or enhance security features in response to emerging threats.

3.2 Proposed System

The proposed system is a **Web Application for Secure Transmission of Data** designed to provide a robust, efficient, and scalable solution for protecting sensitive information exchanged over the internet. The system employs a hybrid encryption approach by integrating Advanced Encryption Standard (AES) for fast and efficient data encryption and Rivest–Shamir–Adleman (RSA) for secure key exchange and user authentication.

The application is developed using the Python Flask framework, offering a lightweight and modular architecture that supports secure session management and easy integration of cryptographic libraries. Users are required to register and authenticate before accessing the system, ensuring only authorized individuals can transmit or receive data. All sensitive information is encrypted before transmission and securely stored in the database.

The system architecture supports end-to-end encryption, ensuring that data remains protected during transmission and storage. Secure database handling is implemented using MySQL with Flask ORM, preventing SQL injection and unauthorized data access. The modular design allows for easy maintenance, scalability, and future security enhancements.

3.2.1 Advantages:

The proposed secure data transmission system offers several advantages over existing solutions. By using a hybrid encryption model, the system achieves both high security and efficient performance. AES enables fast encryption of large data volumes, while RSA ensures secure key exchange and authentication, effectively protecting against common cyber threats.

The system provides strong user authentication and access control, reducing the risk of unauthorized access. Secure session management and encrypted data storage further enhance system security. Additionally, the user-friendly web interface allows users to securely transmit data without requiring technical expertise.

CHAPTER – 4

REQUIREMENTANALYSIS

4.1 Hardware Requirements

- **Processor:** Intel i5 or Above
- **Ram:** Minimum 512 Mb
- **Secondary storage:** Minimum 40 GB

4.2 Software Requirements

- **Operating System:** Windows
- **Programming Language:** Python

CHAPTER - 5
SOFTWARE DESIGN

5.1 System Architecture

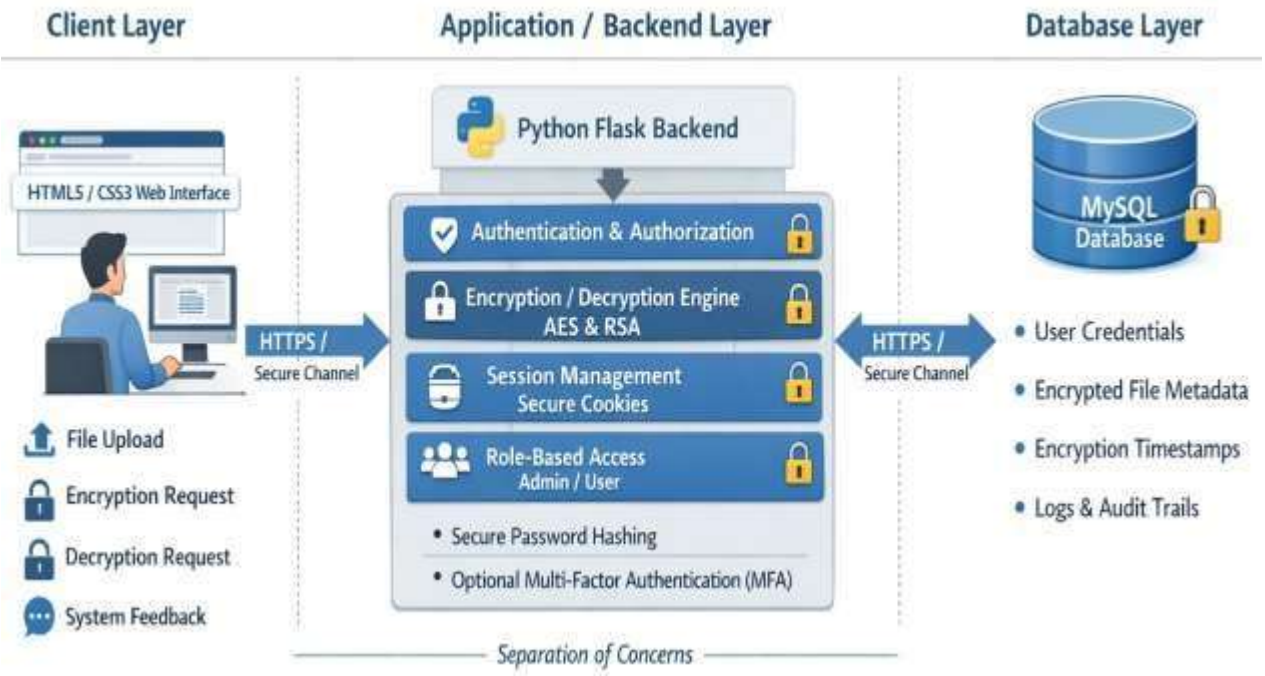


Fig 5.1 System Architecture

5.2 UML Diagrams

5.2.1 Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely.

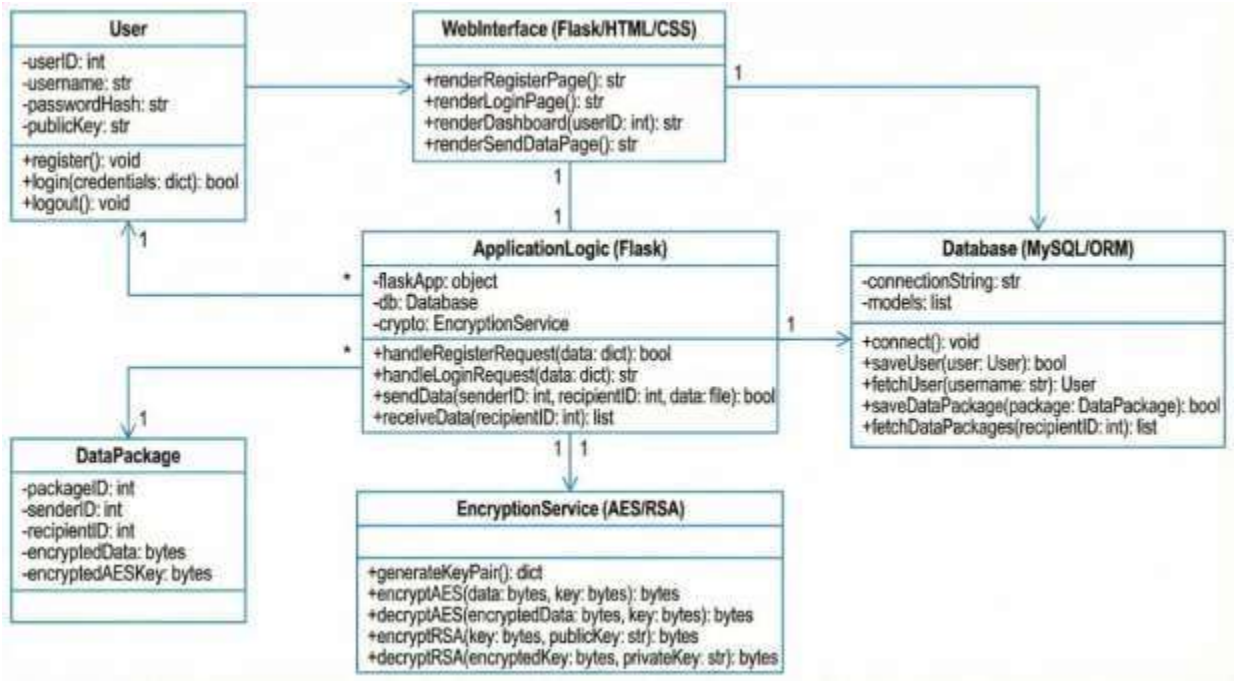


Fig 5.2 Class Diagram

5.2.2 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

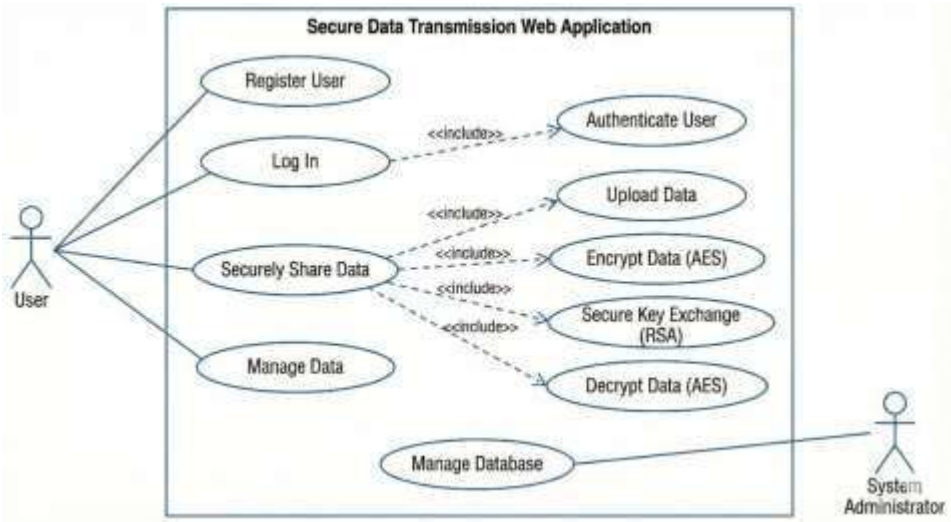


Fig 5.3 Use Case Diagram

5.2.3 Sequence Diagram

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".

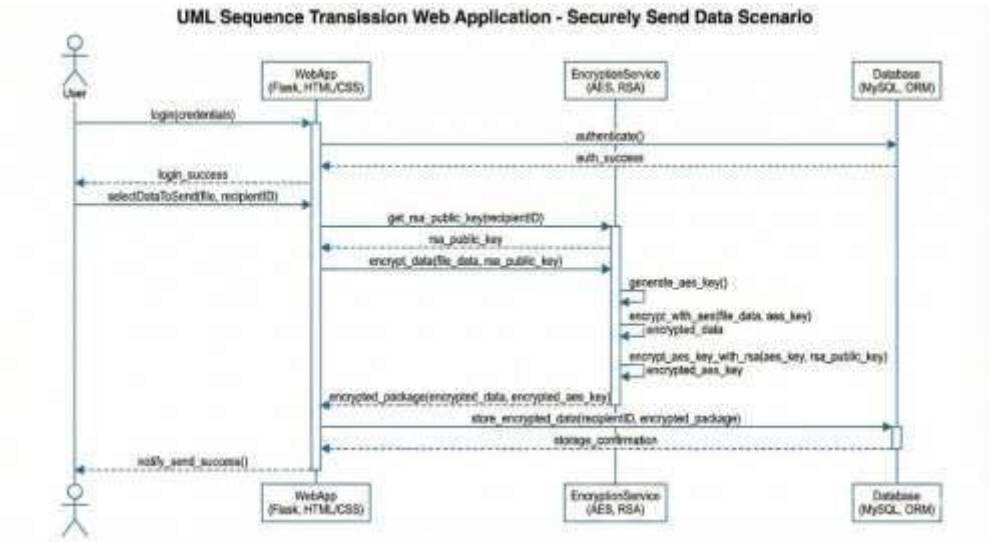


Fig 5.4 Sequence Diagram

CHAPTER - 6
SYSTEM ENVIRONMENT

6.1.1 What is Python

Below are some facts about Python.

- Python is currently the most widely used multi-purpose, high-level programming language.
- Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.
- Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Advantages of Python :-

1. Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. (like c)

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object- oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere

(WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

6.1.2 Advantages of Python over Other Languages

1. **Less Coding**
   - Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.
2. **Affordable**
   - Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.
   - The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.
3. **Python is for Everyone**
   - Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

6.1.3 Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. **Speed Limitations**
   - We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project.

2. **Weak in Mobile Computing and Browsers**
   - While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.
   - The reason it is not so famous despite the existence of Brython is that it isn't that secure.
3. **Design Restrictions**
   - As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.
4. **Underdeveloped Database Access Layers**
   - Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.
5. **Simple**
   - No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.
   - This was all about the Advantages and Disadvantages of Python Programming Language.

6.1.4 History of Python

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python.Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners1, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI).

I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it."Later on in the same Interview, Guido van Rossum continued: "I remembered all

my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers.

Install Python Step-by-Step in Windows or MAC

- Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.
- The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

- There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.
- Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.
- Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e.
- operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here.The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step-1: Go to the official site and use Google Chrome or any other web browser to download and install Python. Or click on the following link:

https://www.python.org/

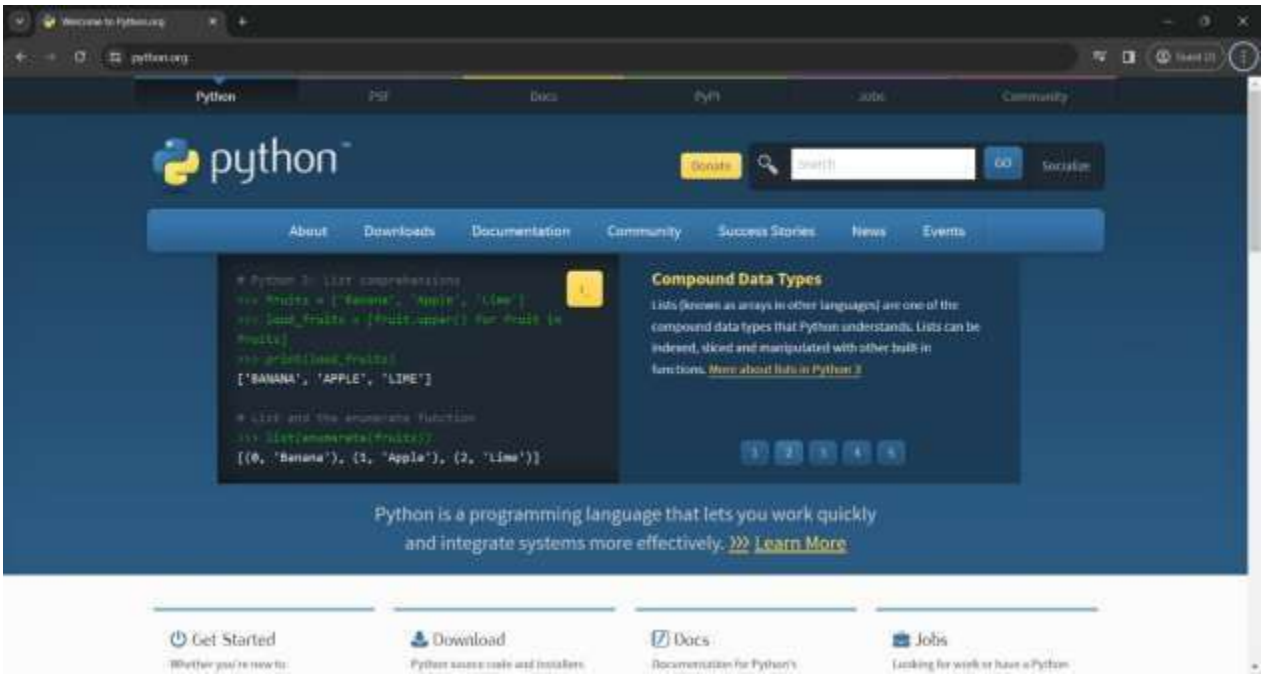


Fig 6.1 Open Python website to download the python latest version

Now check the latest and correct version of your operating system.

Step-2: Click on the Download Tab.

Fig 6.2 Click on Download Python

Step-3: You can select the yellow Download Python for Windows3.7.4 button, or you can scroll download click the download of the corresponding version. Here, we are downloading the latest version of Python for Window 3.7.4.

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Fig 6.3 Select the version what you want

Step-4: Scroll down the page until you find the "File" option.

Step5: Here you will see different versions of Python and operating systems.

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		68111671e5b2db4aef7b9ab01bf0f9be	22.0 MB	SIG
XZ compressed source tarball	Source release		d33e4aae66097051c2eca45ee3604803	16.3 MB	SIG
macOS 64-bit installer	macOS	for OS X 10.9 and later	5dd605c38217a45773bf5e4a936b241f	26.8 MB	SIG
macOS 64-bit/32-bit installer	macOS	for Mac OS X 10.6 and later	6428b4fa7583daff1a442cba8cee08e6	33.3 MB	SIG
Windows help file	Windows		d63999573a2c06b2ac56cade6b4f7cd2	7.8 MB	SIG
Windows x86 embeddable zip file	Windows		9fab3b81f88418796da94133574139d8	6.4 MB	SIG
Windows x86 executable installer	Windows		33cc602942a54446a3d6451476394789	24.5 MB	SIG
Windows x86 web-based installer	Windows		1b676cfa5d317df82c30983ea371d87c	1.3 MB	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b00c8cf6d9ec0b9abe83184a40729a2	7.2 MB	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76debdb3043a583e563400	25.4 MB	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28cb1c608bbd73ae8e53a3bd351b4bd2	1.3 MB	SIG

Fig 6.4 Python Versions

- To download Windows 32bit Python, you can select the built-in Windows X86 Zip file, WindowsX86 executable installer or WindowsX86 installer on the Web.
- TodownloadWindows64 bit Python, you can select any option of the three options. Zip File Embeddable Windows X866, Windows X8664 executable installer or Windows X8664 installer based on the web.

Here you will install the installer based on the Windows X8664 website. Here, the first part of the Python version was completed. Now we will go in advance the second part when installing PythonI. Note: You can click on the option of the release of the version to know the changes or updates made in the version.

Installation of Python

Step-1: Goto Download and Open the downloaded python version to carry out the installation process.

Step-2: Before you click on Install Now, make sure to put a tick on Add Python 3.7 to Path

Fig 6.5 Click on ok to install Python

Fig 6.6 Click on Install Now to install

Step-3: Click on Install Now After the installation is successful. Click on Close.

Fig 6.7 After installation successful. Click on Close

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step-1: Click on Start

Step-2: In the Windows Run Command, type "cmd".

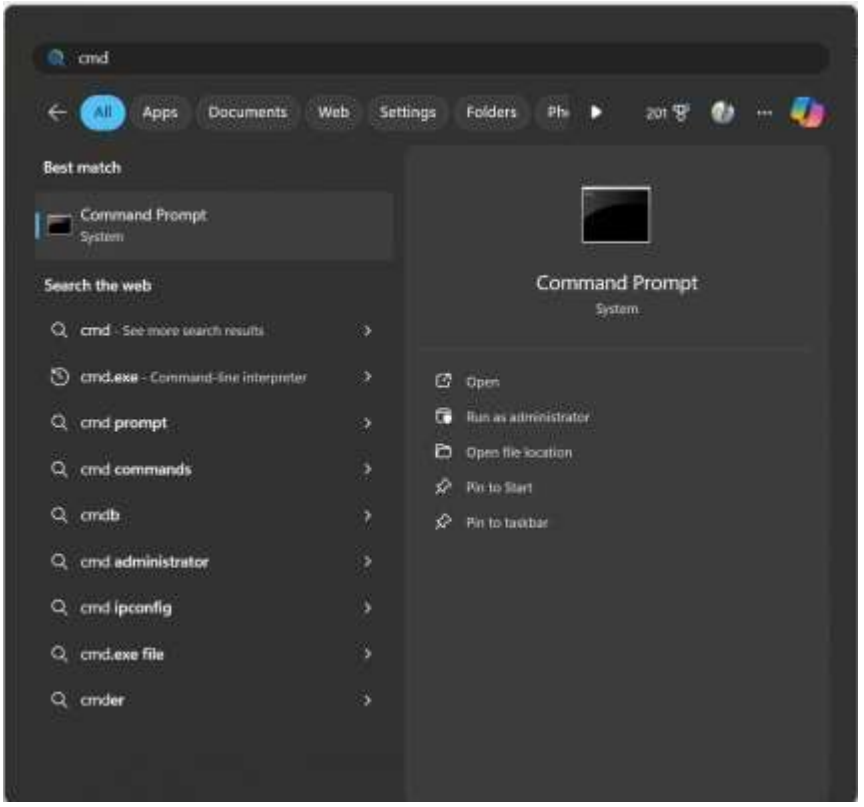


Fig 6.8 Open Command Prompt

Step-3: Open the Command prompt option.

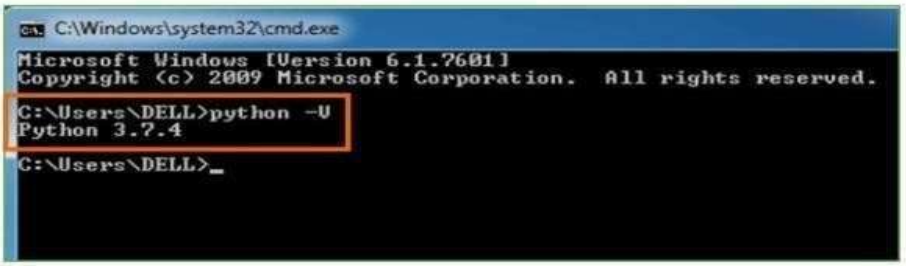


Fig 6.9 Type Python -V to check the python is correctly installed or not

Step-4: Let us test whether the python is correctly installed. Type **python– V** and press Enter.

Step-5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

Step-1: Click on Start

Step-2: In the Windows Run command, type "python idle".

Fig 6.10 Click on IDLE (Python3.7 64-bit) and launch the program

Step-3: Click on IDLE (Python3.7 64-bit) and launch the program

Step-4: To go ahead with working in IDLE you must first save the file. Click on File Click on Save

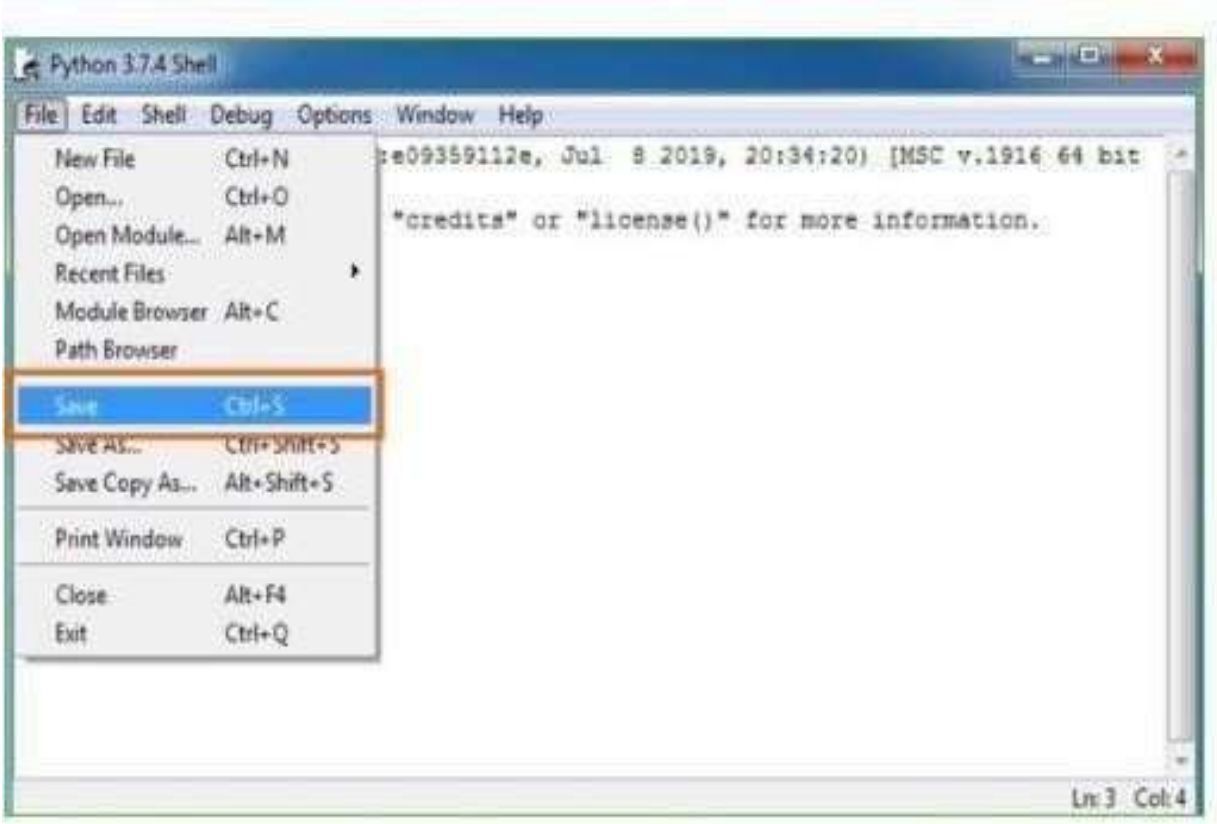


Fig 6.11 Click on File and Save the file

Step-5: Name the file and save as type should be Python files.

Click on SAVE. Here I have named the files as Hey World.

Step-6: Now for e.g. enter print

Python Development Steps

- Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked.Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode.Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x.

- The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python:

"There should be one -- and preferably only one-- obvious way to do it."Some changes in Python 7.3

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behaviour.
- Text Vs. Data Instead of Unicode Vs. 8-bit

Purpose:-

- We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

- Python is an interpreted high-level programming language for general- purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.
- Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.
- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another

area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background- without breaking.

6.2 Flask Web Framework

Flask is a lightweight and flexible **Python-based web framework** used for developing web applications quickly and efficiently. It follows the **Model–View–Controller (MVC)** architectural pattern and provides essential tools and libraries required to build secure and scalable web applications. Due to its simplicity and extensibility, Flask is widely used in academic projects and real-world applications.

In the proposed system, Flask acts as the **backend server** responsible for handling client requests, managing user authentication, processing encryption and decryption operations, and interacting with the database. It serves as the central controller that connects the frontend interface with backend logic and security mechanisms.

Role of Flask in the Project

- Handles **HTTP requests and responses** between users and the system
- Manages **user registration, login, and session handling**
- Integrates **AES and RSA encryption algorithms** for secure data transmission
- Controls routing and API endpoints for secure data access
- Connects with the **MySQL database** to store and retrieve encrypted data
- Ensures secure communication using Flask's built-in security features

6.3 HTML and CSS

HTML (HyperText Markup Language) and CSS (Cascading Style Sheets) are the core technologies used for designing and developing the frontend interface of the web application. In this project, HTML is used to structure the web pages, while CSS is used to control the presentation, layout, and visual appearance of the application.

Role of HTML in the Project

HTML is responsible for creating the structure and content of the web application. It defines elements such as forms, buttons, text fields, and navigation components that allow users to interact with the system.

In the proposed system, HTML is used to:

- Design user registration and login forms
- Create pages for data upload and secure transmission
- Display encrypted and decrypted data
- Provide user-friendly navigation across the application

Role of CSS in the Project

CSS is used to enhance the visual design and user experience of the web application. It controls colors, fonts, layouts, spacing, and responsiveness, ensuring the application is visually appealing and easy to use.

In this project, CSS is used to:

- Design a clean and responsive user interface
- Style forms, buttons, and input fields
- Maintain consistent layout across all web pages
- Improve readability and accessibility

HTML and CSS play a crucial role in developing the frontend of the secure data transmission web application. Together, they provide a structured, responsive, and visually appealing interface that allows users to securely interact with the system while ensuring ease of use and consistency.

6.4 MySQL Database

MySQL is an open-source **relational database management system (RDBMS)** widely used for storing and managing structured data efficiently. In this project, MySQL serves as the **backend database** that securely stores user information, encrypted data, and transaction details required for secure data transmission.

The database is designed to work seamlessly with the Flask web framework, enabling fast and reliable data storage, retrieval, and management while maintaining data integrity and security.

Role of MySQL in the Project

In the proposed system, MySQL is used to:

- Store **user registration and authentication details**
- Save **encrypted data** before transmission
- Maintain **key management information** (public keys, encrypted session keys)
- Track **data transmission logs**
- Support secure and efficient data retrieval

Database Design Approach

The database follows a **normalized relational structure**, reducing redundancy and improving data consistency. Each table is designed with primary keys and foreign key relationships to ensure proper data linkage and integrity.

Key Tables Used in the Project

1. **User Table**
    - Stores user credentials and authentication details
    - Attributes include user ID, username, email, encrypted password, and role
2. **Encrypted Data Table**
    - Stores encrypted files or messages
    - Includes sender ID, receiver ID, encrypted content, timestamp, and status
3. **Key Management Table**
    - Stores public keys and encrypted symmetric keys
    - Helps in secure key exchange using RSA

4. **Transaction / Log Table**
    - Maintains records of data transmission activities
    - Used for auditing and monitoring system activity

Features of MySQL Used

- **Data Integrity** through primary and foreign keys
- **Efficient Query Processing** for fast data access
- **Security Mechanisms** such as access control and encrypted passwords
- **Scalability** to handle multiple users and data records

Advantages of Using MySQL

- Open-source and cost-effective
- High performance and reliability
- Easy integration with Flask using MySQL connectors
- Supports large datasets efficiently
- Well-suited for secure web applications

6.5 Cryptography Overview

Cryptography is the science of securing information by transforming readable data (plaintext) into an unreadable format (ciphertext) using mathematical algorithms. It plays a crucial role in protecting sensitive data during storage and transmission over unsecured networks such as the internet. In modern web applications, cryptography ensures **confidentiality, integrity, authentication, and non-repudiation** of data.

In the proposed **Web Application for Secure Transmission of Data**, cryptographic techniques are used to protect user information and transmitted data from unauthorized access, eavesdropping, and cyber-attacks. The system employs a **hybrid cryptographic approach** by combining symmetric and asymmetric encryption methods to achieve both security and efficiency.

Objectives of Cryptography in the Project

- Protect sensitive data during transmission
- Prevent unauthorized access to confidential information
- Ensure data integrity and authenticity

- Secure encryption key exchange
- Safeguard user credentials and session data

Types of Cryptography Used

1. Symmetric Key Cryptography

Symmetric cryptography uses a **single secret key** for both encryption and decryption. It is highly efficient and suitable for encrypting large amounts of data.

Algorithm Used:

- **Advanced Encryption Standard (AES)**

Role in the Project:

- Encrypts user data before transmission
- Ensures fast and secure data encryption
- Protects data stored in the database

2. Asymmetric Key Cryptography

Asymmetric cryptography uses a **pair of keys** — a public key and a private key. It is primarily used for secure key exchange and authentication.

Algorithm Used:

- **Rivest–Shamir–Adleman (RSA)**

Role in the Project:

- Securely exchanges AES keys
- Provides authentication between communicating users
- Prevents man-in-the-middle attacks

Hybrid Cryptographic Approach

The proposed system uses a **hybrid encryption technique**, combining the strengths of both symmetric and asymmetric cryptography:

- RSA encrypts and securely shares the AES secret key
- AES encrypts the actual data for transmission
- Improves performance and security

- Reduces computational overhead
- This approach ensures that even if communication is intercepted, the data remains protected and unreadable.

Security Properties Achieved

- **Confidentiality:** Only authorized users can access data
- **Integrity:** Data remains unaltered during transmission
- **Authentication:** Ensures legitimate users communicate
- **Non-repudiation:** Prevents denial of data transmission

Importance of Cryptography in the Project

With increasing cyber threats such as data breaches, phishing, and unauthorized access, cryptography is essential for secure digital communication. By integrating AES and RSA algorithms into a Flask-based web application, the proposed system provides a robust security solution for transmitting sensitive information over the web.

6.5.1 AES Algorithm

The Advanced Encryption Standard (AES) is a widely used **symmetric key encryption algorithm** designed to secure sensitive data. It was established by the National Institute of Standards and Technology (NIST) and is recognized for its strong security, efficiency, and reliability. AES is extensively used in web applications, financial systems, and secure communication platforms.

In the proposed **Web Application for Secure Transmission of Data**, AES is used to encrypt user data before transmission and storage. Due to its high speed and low computational overhead, AES is suitable for encrypting large volumes of data efficiently.

Key Features of AES

- Symmetric encryption using a single secret key
- Fixed block size of **128 bits**
- Supports key sizes of **128, 192, and 256 bits**

- Highly secure and resistant to known cryptographic attacks
- Efficient for both software and hardware implementations

Working of AES Algorithm

AES operates on data blocks through a series of transformation rounds. The number of rounds depends on the key size:

Key Size	Number of Rounds
128-bit	10
192-bit	12
256-bit	14

Each AES round consists of the following steps:

1. **SubBytes**
   Each byte of the data block is substituted using an S-Box to introduce non-linearity.
2. **ShiftRows**
   Rows of the data block are shifted cyclically to improve diffusion.
3. **MixColumns**
   Columns are mixed using mathematical operations to further scramble the data.
4. **AddRoundKey**
   The data block is combined with the round key using XOR operation.

The final round omits the MixColumns step.

AES Encryption Process in the Project

1. User submits sensitive data through the web interface.
2. Flask backend generates or retrieves a secret AES key.
3. Data is encrypted using AES before transmission.
4. Encrypted data is stored securely in the MySQL database.
5. Only authorized users with the correct key can decrypt the data.

Advantages of AES in the Project

- Fast and efficient encryption
- Strong resistance against brute-force attacks
- Suitable for real-time secure web applications
- Low computational cost compared to asymmetric algorithms
- Ensures data confidentiality and integrity

Limitations of AES

- Requires secure key distribution
- Key compromise leads to data exposure
- Needs integration with asymmetric encryption for key exchange

To overcome these limitations, the proposed system integrates **RSA** to securely exchange AES keys.

Conclusion

AES plays a critical role in ensuring secure data transmission in the proposed web application. By encrypting sensitive information efficiently and securely, AES enhances system performance while maintaining strong data protection.

When combined with RSA in a hybrid encryption approach, AES provides a reliable and robust security solution for modern web-based applications.

6.5.2 RSA Algorithm

RSA (Rivest–Shamir–Adleman) is a widely used **asymmetric key cryptographic algorithm** that provides secure data transmission and authentication over unsecured networks. Unlike symmetric encryption, RSA uses **two different keys**: a **public key** for encryption and a **private key** for decryption. This key-pair mechanism makes RSA highly suitable for secure key exchange and digital authentication.

In the proposed **Web Application for Secure Transmission of Data**, RSA is used to securely exchange AES secret keys and authenticate users. This ensures that encryption keys are shared safely without being exposed to unauthorized users.

Key Features of RSA

- Asymmetric encryption using public and private keys
- Strong mathematical foundation based on large prime numbers
- Secure key exchange over insecure networks
- Widely used for authentication and digital signatures
- High level of security when using large key sizes

Working of RSA Algorithm

The RSA algorithm operates in three main phases:

1. Key Generation

- Two large prime numbers are selected.
- Public and private keys are mathematically generated.
- The public key is shared openly, while the private key is kept secret.

2. Encryption

- The sender encrypts data or a secret key using the recipient's public key.
- Encrypted data can only be decrypted using the corresponding private key.

3. Decryption

- The receiver uses the private key to decrypt the encrypted data.

RSA Key Sizes

Key Size	Security Level
1024-bit	Basic (not recommended)
2048-bit	Standard (widely used)
4096-bit	High security

The proposed system uses **2048-bit RSA keys** for a balance between security and performance.

Role of RSA in the Project

- Secure exchange of AES encryption keys
- User authentication and identity verification
- Protection against man-in-the-middle attacks
- Ensures confidentiality and integrity of key transmission

Advantages of RSA

- Secure key distribution without prior shared secrets
- High resistance to brute-force attacks
- Enables authentication and secure communication
- Trusted and widely adopted encryption standard

Limitations of RSA

- Slower than symmetric algorithms like AES
- Not suitable for encrypting large data volumes
- Requires higher computational power

To address these limitations, the proposed system combines RSA with AES in a **hybrid encryption model**, where RSA handles key exchange and AES encrypts actual data.

Conclusion

RSA plays a vital role in the proposed web application by ensuring secure key exchange and authentication. When integrated with AES, RSA strengthens the overall security architecture of the system, making it resistant to common cyber threats while maintaining efficient performance.

CHAPTER – 7

MODULES USED IN PROJECT

7.1 User Registration and Authentication Module

The **User Registration and Authentication Module** is a fundamental component of the *Web Application for Secure Transmission of Data*, responsible for ensuring that only authorized users can access the system and transmit sensitive information securely. During the registration process, users are required to provide essential details such as username, email address, and password. To enhance security, user passwords are not stored in plain text; instead, they are securely hashed before being saved in the MySQL database. This prevents unauthorized access even in the event of a data breach.

The authentication process verifies user credentials during login by comparing the entered information with the securely stored records in the database. Upon successful authentication, a secure user session is created using Flask's session management, allowing users to access protected features of the application. Unauthorized users are restricted from accessing secure data transmission functionalities. This module also helps prevent common security threats such as brute-force attacks and unauthorized access by ensuring proper validation and session handling. Overall, the User Registration and Authentication Module plays a critical role in maintaining data confidentiality, system integrity, and secure communication within the application.

7.2 Encryption and Decryption Module

The **Encryption and Decryption Module** is a core security component of the *Web Application for Secure Transmission of Data*, designed to protect sensitive information during transmission and storage. This module uses a **hybrid cryptographic approach** by integrating the Advanced Encryption Standard (AES) and the Rivest–Shamir–Adleman (RSA) algorithms. Before data is transmitted or stored, it is encrypted using AES, which provides fast and efficient symmetric encryption for large volumes of data. To ensure secure key exchange, the AES secret key is encrypted using RSA and shared only with authorized users, preventing key exposure over unsecured networks.

During data retrieval, the module performs the decryption process by first securely decrypting the AES key using the recipient's RSA private key and then using that key to decrypt the actual data. This ensures that only authenticated and authorized users can access the original information. By encrypting data end-to-end, the module safeguards against common security threats such as eavesdropping, data tampering,

and man-in-the-middle attacks. Overall, the Encryption and Decryption Module plays a vital role in maintaining confidentiality, integrity, and trust in the secure data transmission system.

7.3 Secure Data Transmission Module

The **Secure Data Transmission Module** is a vital component of the *Web Application for Secure Transmission of Data*, responsible for ensuring that sensitive information is transmitted safely between users over the internet. This module works in coordination with the authentication and encryption components to guarantee that data is shared only between authorized parties. Before transmission, all data is encrypted using the Advanced Encryption Standard (AES), and the encryption keys are securely exchanged using the RSA algorithm. This ensures that even if the data is intercepted during transmission, it remains unreadable and protected from unauthorized access.

The module manages the secure transfer of encrypted data through the Flask web server, maintaining data confidentiality and integrity throughout the communication process. It also ensures proper session validation so that data is transmitted only during authenticated user sessions. Secure transmission logs and status updates are maintained to monitor successful data transfers and detect any anomalies. By implementing end-to-end encryption and controlled access, the Secure Data Transmission Module effectively protects against common cyber threats such as eavesdropping, replay attacks, and man-in-the-middle attacks, making the system reliable and trustworthy for secure digital communication.

7.4 Database Management Module

The **Database Management Module** plays a crucial role in the *Web Application for Secure Transmission of Data* by securely storing and managing all application-related data. This module uses a MySQL database to handle user information, encrypted data, encryption keys (in protected form), access logs, and transaction records. Sensitive information such as passwords is never stored in plain text; instead, it is securely hashed, while transmitted data is stored only in its encrypted format. This approach ensures data confidentiality even in the event of unauthorized database access.

In addition, the Database Management Module supports efficient data retrieval, update, and deletion operations while enforcing strict access control policies.

It integrates seamlessly with the Flask backend to validate user credentials, manage session data, and track secure data transmissions. Regular database integrity checks and structured relational design improve performance and reliability. By combining secure storage practices with controlled access and efficient data handling, this module ensures that the application maintains high standards of data security, consistency, and scalability.

CHAPTER – 8

TESTING

8.1 System Test

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 Types Of Tests

8.2.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.2.2 Integrated Testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

8.2.3 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

1. **Valid Input:** Identified classes of valid input must be accepted.
2. **Invalid Input:** Identified classes of invalid input must be rejected
3. **Functions:** Identified functions must be exercised.
4. **Output:** Identified classes of application outputs must be exercised.
5. **Systems/Procedures:** Interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

8.2.4 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure, and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

8.2.5 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

8.2.6 Unit Testing

- Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

- Test strategy and approach
- Field testing will be performed manually and functional tests will be written in detail.
- Test objectives
- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.
- Features to be tested
- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.
- Integration Testing
- Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.
- The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.
- Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

- User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.
- Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Test Case – 1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test Case – 2:

Test case for User Registration form:

FUNCTION:	USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test Case – 3:

- Test case for Change Password:
- When the old password does not match with the new password ,then this results in displaying an error message as " OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD".

FUNCTION:	Change Password
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

CHAPTER - 9

IMPLEMENTATION

9.1 Methodology

The **Web Application for Secure Transmission of Data** is implemented using a combination of modern web technologies and cryptographic techniques to ensure confidentiality, integrity, and secure communication between users. The system follows a client–server architecture where sensitive data is encrypted before transmission and securely decrypted at the receiver's end.

1. System Architecture

The application is based on a **three-tier architecture**:

1. **Presentation Layer (Frontend)**
   - Provides user interfaces for registration, login, data upload, and data access.
   - Ensures secure user interaction through authentication mechanisms.
2. **Application Layer (Backend)**
   - Handles business logic, encryption, decryption, and access control.
   - Manages user authentication and secure data processing.
3. **Data Layer (Database)**
   - Stores encrypted data and user credentials.
   - Prevents unauthorized access to sensitive information.

2. Technology Stack Used

- **Frontend:** HTML, CSS
- **Backend:** Python (Flask)
- **Database:** MySQL
- **Security Algorithms:**
  - AES (Advanced Encryption Standard) for data encryption
  - RSA for secure key exchange
  - SHA-256 for password hashing
- **Protocol:** HTTPS

3. User Authentication Module

- Users must register with valid credentials.
- Passwords are **hashed using SHA-256** before storing in the database.
- Login credentials are verified securely.

Implementation Steps:

1. User submits registration details.
2. Password is hashed before storage.
3. During login, entered password is hashed and compared.
4. Secure session is created upon successful authentication.

4. Data Encryption Module

- Before data transmission, the system encrypts data using **AES symmetric encryption**.
- A unique encryption key is generated for each session.
- The encryption key is securely shared using **RSA public key encryption**.

Process Flow:

1. User enters data to be transmitted.
2. Data is encrypted using AES.
3. AES key is encrypted using receiver's RSA public key.
4. Encrypted data and encrypted key are sent to the server.

5. Secure Data Transmission

- All communication between client and server occurs over **HTTPS**.
- Encrypted payload ensures data confidentiality even if intercepted.
- Integrity is maintained by verifying decrypted content at the receiver's end.

6. Data Decryption Module

- Authorized users can access encrypted data.
- The encrypted AES key is first decrypted using the receiver's RSA private key.
- The data is then decrypted using the recovered AES key.

Steps:

1. Receiver requests encrypted data.
2. RSA private key decrypts the AES key.
3. AES decrypts the data.
4. Original data is displayed securely.

7. Database Security

- Only encrypted data is stored in the database.
- Access control mechanisms restrict unauthorized database access.

- Regular validation and input sanitization prevent SQL injection attacks.

8. Error Handling and Logging

- Secure error handling avoids exposure of sensitive system details.
- Logs are maintained for authentication attempts and data access.
- Failed access attempts are monitored for security analysis.

9. Performance and Scalability

- Encryption is optimized to minimize transmission delay.
- Modular design allows easy integration of advanced encryption algorithms.
- The system supports multiple concurrent users securely.

10. Result and Outcome

The implemented web application successfully ensures:

- Secure authentication of users
- Encrypted data transmission over the network
- Protection against unauthorized access and data breaches
- Confidentiality, integrity, and reliabilityuthentication of transmitted data

9.2 Source Code

app.py:

```
from flask import Flask, render_template, request, redirect, url_for, session, send_file, flash, abort
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
import os
import io
import zipfile
import secrets
import string
from database import init_db, add_user, get_user, save_file_metadata, get_file_metadata,
get_user_files
from encryption.encrypt import encrypt_file
from encryption.decrypt import decrypt_file
from encryption.key_utils import generate_key_from_password
from encryption.rsa_utils import generate_rsa_key_pair, rsa_encrypt, rsa_decrypt

app = Flask(__name__)
app.secret_key = 'supersecretkey_change_this_for_prod'

UPLOAD_FOLDER = os.path.join(os.getcwd(), 'uploads')
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)


init_db()

def generate_share_id(length=6):
    """Generates a short unique ID for sharing."""
    chars = string.ascii_uppercase + string.digits
    return ''.join(secrets.choice(chars) for _ in range(length))

@app.route('/', methods=['GET', 'POST'])
def auth():
    if 'user_id' in session:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        action = request.form.get('action')

        if action == 'register':
            username = request.form['username']
            email = request.form['email']
            password = request.form['password']

            hashed_pw = generate_password_hash(password)
            if add_user(username, email, hashed_pw):
                flash('Account created! Please login.', 'success')
```

```
            return redirect(url_for('auth', mode='login'))
        else:

flash('Username already exists or Database Error.', 'error')
            return redirect(url_for('auth', mode='register'))

    elif action == 'login':
        username = request.form['username']
        password = request.form['password']
        user = get_user(username)
        if user and check_password_hash(user['password'], password):
            session['user_id'] = user['id']
            session['username'] = user['username']
            return redirect(url_for('dashboard'))
        else:
            flash('Invalid credentials.', 'error')

  mode = request.args.get('mode', 'login')
  return render_template('auth.html', mode=mode)

@app.route('/logout')
def logout():
  session.clear()
  return redirect(url_for('auth'))

@app.route('/dashboard')
def dashboard():
  if 'user_id' not in session: return redirect(url_for('auth'))


  my_files = get_user_files(session['user_id'])
  return render_template('dashboard.html', username=session['username'], files=my_files)



@app.route('/aes')
def aes_options():
  if 'user_id' not in session: return redirect(url_for('auth'))
  return render_template('aes_options.html')

@app.route('/aes/encrypt', methods=['GET', 'POST'])
def aes_encrypt():
  if 'user_id' not in session: return redirect(url_for('auth'))

  if request.method == 'POST':
    file = request.files.get('file')
    password = request.form.get('password')

    if not file or file.filename == '' or not password:
        flash('File and Password required', 'error')
        return redirect(request.url)
```

```
        try:
            filename = secure_filename(file.filename)
            file_data = file.read()


            key = generate_key_from_password(password)
            encrypted_data = encrypt_file(file_data, key)

            share_id = generate_share_id()
            enc_filename = f"{share_id}_{filename}.enc"
            filepath = os.path.join(UPLOAD_FOLDER, enc_filename)

            with open(filepath, 'wb') as f:
                f.write(encrypted_data)


            save_file_metadata(filename, filepath, session['user_id'], key, share_id)


            flash(f'File Uploaded Successfully!', 'success')
            return render_template('aes_options.html', share_id=share_id, filename=enc_filename)

        except Exception as e:
            flash(f'Encryption failed: {str(e)}', 'error')
            return redirect(request.url)

    return render_template('aes_encrypt.html')

@app.route('/download/<share_id>')
def download_encrypted(share_id):
    if 'user_id' not in session: return redirect(url_for('auth'))

    file_record = get_file_metadata(share_id)
    if not file_record:
        abort(404)

    try:
        return send_file(
            file_record['file_path'],
            as_attachment=True,
            download_name=os.path.basename(file_record['file_path'])
        )
    except FileNotFoundError:
        abort(404)

@app.route('/aes/decrypt', methods=['GET', 'POST'])
def aes_decrypt():
    if 'user_id' not in session: return redirect(url_for('auth'))

    if request.method == 'POST':

        file = request.files.get('file')
```

```
    password = request.form.get('password')
    share_id = request.form.get('share_id')

    if share_id:

        file_record = get_file_metadata(share_id)
        if not file_record:
            flash('Invalid Share ID', 'error')
            return redirect(request.url)

        try:
            with open(file_record['file_path'], 'rb') as f:
                file_data = f.read()
            filename = file_record['original_filename']
        except FileNotFoundError:
            flash('File not found on server', 'error')
            return redirect(request.url)

    elif file and file.filename != '':

        filename = secure_filename(file.filename)
        file_data = file.read()
    else:
        flash('Please upload a file OR enter a Share ID', 'error')
        return redirect(request.url)

    if not password:
        flash('Password required', 'error')
        return redirect(request.url)

    try:
        key = generate_key_from_password(password)
        decrypted_data = decrypt_file(file_data, key)

        download_name = filename.replace('.enc', '') if filename.endswith('.enc') else 'decrypted_' +
filename

        return send_file(
            io.BytesIO(decrypted_data),
            download_name=download_name,
            as_attachment=True
        )

    except Exception as e:
        flash('Decryption Failed: Incorrect Password or Corrupt File.', 'error')
        return redirect(request.url)

  return render_template('aes_decrypt.html')


@app.route('/rsa')
```

```
def rsa_options():
    if 'user_id' not in session: return redirect(url_for('auth'))

return render_template('rsa_options.html')

@app.route('/rsa/generate')
def rsa_generate():
    if 'user_id' not in session: return redirect(url_for('auth'))

    private_pem, public_pem = generate_rsa_key_pair()


    memory_file = io.BytesIO()
    with zipfile.ZipFile(memory_file, 'w') as zf:
        zf.writestr('private_key.pem', private_pem)
        zf.writestr('public_key.pem', public_pem)
    memory_file.seek(0)

    return send_file(
        memory_file,
        download_name='rsa_keys.zip',
        as_attachment=True
    )

@app.route('/rsa/encrypt', methods=['GET', 'POST'])
def rsa_encrypt_view():
    if 'user_id' not in session: return redirect(url_for('auth'))

    if request.method == 'POST':
        file = request.files.get('file')
        pub_key_file = request.files.get('public_key')

        if not file or not pub_key_file:
            flash('Files required', 'error')
            return redirect(request.url)

        try:
            filename = secure_filename(file.filename)
            file_data = file.read()
            public_key_pem = pub_key_file.read()


            encrypted_data = rsa_encrypt(file_data, public_key_pem)


            share_id = generate_share_id()
            enc_filename = f"{share_id}_{filename}.rsa.enc"
            filepath = os.path.join(UPLOAD_FOLDER, enc_filename)

            with open(filepath, 'wb') as f:
                f.write(encrypted_data)
```

```
            # For RSA Hybrid, the shared_key blob could store the Encrypted AES Key (first 256 bytes)
            # But currently `rsa_encrypt` returns (EncryptedKey + EncryptedData) as one blob.
            # We'll just store a dummy or valid marker in shared_key, or storing Public Key?
            # Storing the Public Key PEM is useful.


            save_file_metadata(filename, filepath, session['user_id'], public_key_pem, share_id)

            flash(f'File Encrypted & Saved! Share ID: {share_id}', 'success')


            return send_file(
                io.BytesIO(encrypted_data),
                as_attachment=True,
                download_name=filename + '.rsa.enc',
                mimetype='application/octet-stream'
            )
        except Exception as e:
            flash(f'Encryption failed: {str(e)}', 'error')
            return redirect(request.url)

    return render_template('rsa_encrypt.html')

@app.route('/rsa/decrypt', methods=['GET', 'POST'])
def rsa_decrypt_view():
    if 'user_id' not in session: return redirect(url_for('auth'))

    if request.method == 'POST':
        file = request.files.get('file')
        priv_key_file = request.files.get('private_key')
        share_id = request.form.get('share_id')

        file_data = None
        filename = "decrypted_file"

        if share_id:

            file_record = get_file_metadata(share_id)
            if not file_record:
                flash('Invalid Share ID', 'error')
                return redirect(request.url)
            try:
                with open(file_record['file_path'], 'rb') as f:
                    file_data = f.read()
                filename = file_record['original_filename']
            except FileNotFoundError:
                flash('File not found', 'error')
                return redirect(request.url)

        elif file:
            filename = secure_filename(file.filename)
            file_data = file.read()
```

```
        if not file_data or not priv_key_file:
            flash('Encrypted File (or ID) and Private Key required', 'error')
            return redirect(request.url)

        try:
            private_key_pem = priv_key_file.read()
            decrypted_data = rsa_decrypt(file_data, private_key_pem)

            download_name = filename.replace('.rsa.enc', '') if filename.endswith('.rsa.enc') else
'decrypted_' + filename

            return send_file(
                io.BytesIO(decrypted_data),
                download_name=download_name,
                as_attachment=True
            )
        except Exception as e:
            flash(f'Decryption failed: {str(e)}', 'error')
            return redirect(request.url)

    return render_template('rsa_decrypt.html')

if __name___== '__main__':
    app.run(debug=True)
```

database.py:

```
import mysql.connector
from datetime import datetime

DB_CONFIG = {
    'host': '127.0.0.1',
    'user': 'root',
    'password': 'root',
    'database': 'Secureshare'
}

def get_db_connection():
    """Create a new database connection."""
    conn = mysql.connector.connect(**DB_CONFIG)
    return conn

def init_db():
    """Initialize the database and tables."""
    try:
        # Connect to MySQL server (without DB) to create DB
        conn = mysql.connector.connect(
            host=DB_CONFIG['host'],
```

```
            user=DB_CONFIG['user'],
            password=DB_CONFIG['password']
        )
        cursor = conn.cursor()
        cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_CONFIG['database']}")
        conn.close()

        # Connect to the DB
        conn = get_db_connection()
        cursor = conn.cursor()

        # Users Table (Already created by user)
        # files Table (Already created by user)
        pass

    except mysql.connector.Error as err:
        print(f"Error checking database: {err}")

def add_user(username, email, password_hash):
    conn = get_db_connection()
    cursor = conn.cursor()
    try:
        # User defined table 'Users' with column 'password_hash'
        cursor.execute("INSERT INTO Users (username, email, password_hash) VALUES (%s, %s,
%s)",
                       (username, email, password_hash))
        conn.commit()
        return True
    except mysql.connector.IntegrityError:
        return False
    except mysql.connector.Error as err:
        print(f"Database Error: {err}")
        return False
    finally:
        cursor.close()
        conn.close()

def get_user(username):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    # User defined table 'Users'
    cursor.execute("SELECT * FROM Users WHERE username = %s", (username,))
    user = cursor.fetchone()
    cursor.close()
    conn.close()

    # Adapt to app's expected keys if necessary
    if user:
        user['password'] = user['password_hash'] # Map for compatibility

    return user
```

```
def save_file_metadata(original_filename, file_path, uploaded_by_id, shared_key, share_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    try:
        # Schema: original_filename, file_path, uploaded_by (int), shared_key (blob), share_id
        cursor.execute(
            "INSERT INTO files (original_filename, file_path, uploaded_by, shared_key, share_id)
VALUES (%s, %s, %s, %s, %s)",
            (original_filename, file_path, uploaded_by_id, shared_key, share_id)
        )
        conn.commit()
    except mysql.connector.Error as err:
        print(f"Error saving file: {err}")
    finally:
        cursor.close()
        conn.close()

def get_file_metadata(share_id):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM files WHERE share_id = %s", (share_id,))
    file = cursor.fetchone()
    cursor.close()
    conn.close()
    return file

def get_user_files(user_id):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM files WHERE uploaded_by = %s ORDER BY uploaded_at
DESC", (user_id,))
    files = cursor.fetchall()

    # Detect Algorithm based on shared_key content
    for file in files:
        key_data = file.get('shared_key')
        if key_data and isinstance(key_data, bytes) and b'-----BEGIN PUBLIC KEY-----' in key_data:
            file['algorithm'] = 'RSA'
        elif key_data and isinstance(key_data, str) and '-----BEGIN PUBLIC KEY-----' in key_data:
            file['algorithm'] = 'RSA'
        else:
            file['algorithm'] = 'AES'

    cursor.close()
    conn.close()
    return files

def update_file_key(share_id, new_key):
    conn = get_db_connection()
    cursor = conn.cursor()
    try:
        cursor.execute(
```

```
            "UPDATE files SET shared_key = %s WHERE share_id = %s",
            (new_key, share_id)
        )
        conn.commit()
    except mysql.connector.Error as err:
        print(f"Error updating file key: {err}")
    finally:
        cursor.close()
        conn.close()

def delete_file(share_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    try:
        cursor.execute("DELETE FROM files WHERE share_id = %s", (share_id,))
        conn.commit()
    except mysql.connector.Error as err:
        print(f"Error deleting file record: {err}")
    finally:
        cursor.close()
        conn.close()
```

CHAPTER – 10

RESULTS

Screenshots:

Fig 10.1 Login Interface

SecureShare user authentication page for authorized access to the secure data transmission system.

Fig 10.2 Algorithm Selection

Fig 10.3 AES Operations Menu

Interface providing options to encrypt or decrypt files using AES encryption.

Fig 10.4 AES Encryption Page

File upload interface for encrypting data using a password-based AES algorithm.

Fig 10.5 Encryption Result

Confirmation screen displaying successful encryption and generated secure share ID.

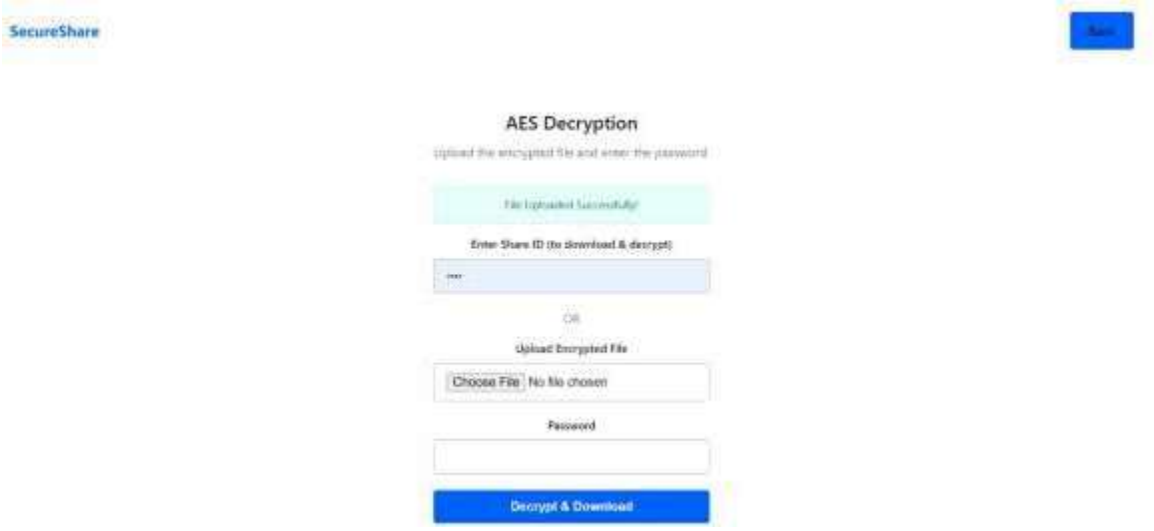


Fig 10.6 AES Decryption Page

Interface for decrypting encrypted files using share ID and password.

Fig 10.7 RSA Operations Dashboard

This dashboard provides RSA cryptographic operations, including key generation, file encryption using a public key, and file decryption using a private key.

Fig 10.8 RSA Encryption Screen

This page enables users to encrypt a file using the recipient's RSA public key, ensuring secure data transmission.

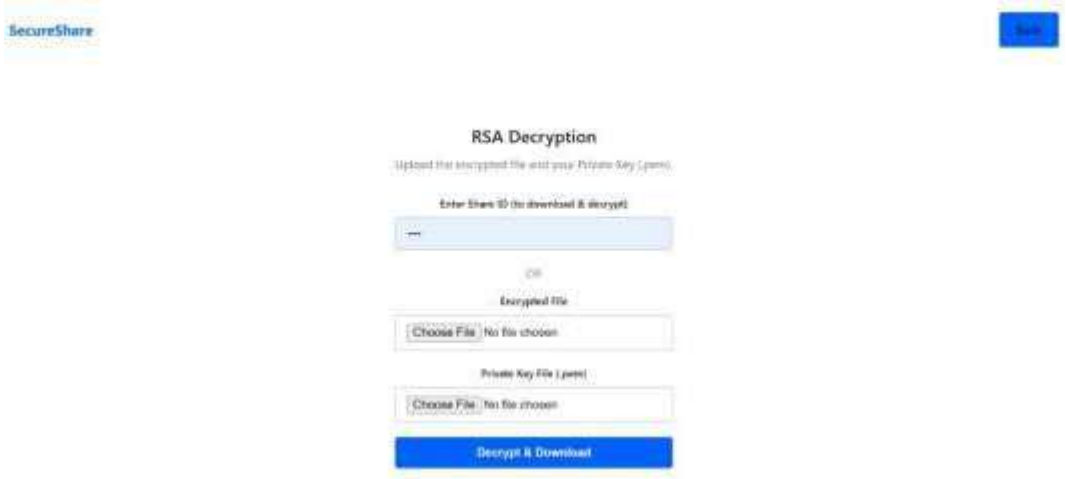


Fig 10.9 RSA Decryption Screen

This screen allows users to decrypt RSA-encrypted files by uploading the encrypted file and the corresponding private key.

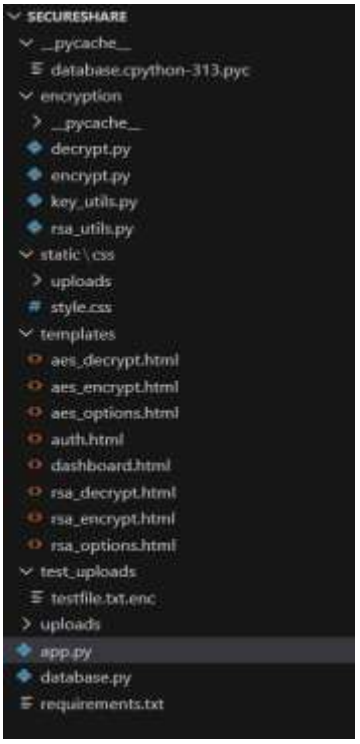


Fig 10.10 Project Folder Structure

CHAPTER – 11

CONCLUSION

In conclusion, the development of the Web Application for Secure Transmission of Data successfully addresses the critical need for protecting sensitive information during online communication. The project demonstrates how modern web technologies combined with strong security mechanisms can ensure data confidentiality, integrity, and authenticity throughout the transmission process. By implementing secure authentication, encryption techniques, and controlled access mechanisms, the system effectively prevents unauthorized access and data breaches.

The application provides a reliable and user-friendly platform that allows users to transmit data securely over the internet while minimizing risks associated with cyber threats such as data interception, tampering, and unauthorized modification. The use of encryption algorithms ensures that data remains protected both during transmission and storage, while secure login and session management enhance overall system security. This makes the application suitable for use in environments where data privacy is crucial, such as business communication, academic data exchange, and secure file sharing.

Overall, the project highlights the importance of secure data transmission in today's digital world and demonstrates a practical implementation of web-based security solutions. It reduces dependence on manual security practices and provides an automated, efficient, and scalable approach to safeguarding information.

Future Work:

Future enhancements to this project can focus on improving security strength and system performance. Advanced encryption standards, multi-factor authentication, and role-based access control can be integrated to further strengthen data protection. The application can also be extended to support secure real-time data transmission, cloud integration, and API-based communication between systems.

Additionally, optimizing the application for mobile devices and incorporating secure key management techniques can enhance usability and scalability. Implementing intrusion detection, activity logging, and audit mechanisms would further improve system reliability and trust. With these enhancements, the web application can evolve into a comprehensive secure communication platform suitable for large-scale and enterprise-level deployment.

CHAPTER – 12

REFERENCES

1. Research Papers & Articles :

- Stallings, W. (2017).

Cryptography and Network Security: Principles and Practice. Pearson Education.

– Provides fundamental concepts of encryption, authentication, and secure communication.

- Diffie, W., & Hellman, M. (1976).

New Directions in Cryptography. IEEE Transactions on Information Theory, 22(6), 644–654.

– Introduces public-key cryptography, which is foundational for secure data transmission.

- Dierks, T., & Rescorla, E. (2008).

The Transport Layer Security (TLS) Protocol Version 1.2. IETF RFC 5246.

– Describes TLS, a core protocol used for secure web communication.

- Rescorla, E. (2018).

The Transport Layer Security (TLS) Protocol Version 1.3. IETF RFC 8446.

– Explains improvements in modern secure web data transmission.

- Kahn Academy & OWASP Foundation (2022).

OWASP Top 10 Web Application Security Risks.

– Identifies common vulnerabilities and best practices for securing web applications.

2. Books & Technical Documentation :

- Kahn Academy & Mozilla Developer Network (MDN)

Web Security Documentation.

https://developer.mozilla.org/en-US/docs/Web/Security

– Covers HTTPS, authentication, encryption, and secure sessions.

- William Stallings (2018).

Network Security Essentials: Applications and Standards. Pearson Education.

– Focuses on secure network communication and encryption techniques.

3. Web Technologies & Frameworks :

- W3C

Web Cryptography API Specification.

https://www.w3.org/TR/WebCryptoAPI/

– Standard for performing cryptographic operations in web applications.

4. Tools, Libraries & Platforms :

- OpenSSL Project

OpenSSL Cryptography and SSL/TLS Toolkit.

https://www.openssl.org/

– Widely used library for implementing secure communication.

https://tomcat.apache.org/security.html

– Describes securing web servers and data transmission.

5. Online Learning Resources :

- Cloudflare Learning Center

What is HTTPS and SSL/TLS?

https://www.cloudflare.com/learning/ssl/what-is-https/

– Explains secure data transmission concepts in simple terms.

- GeeksforGeeks

Secure Data Transmission Techniques in Web Applications.

https://www.geeksforgeeks.org

– Provides practical examples and explanations of encryption and secure communication.

PUBLISHED PAPER & PAPER PUBLICATION CERTIFICATES

iJRASET
International Journal For Research in Applied Science and Engineering Technology

INTERNATIONAL JOURNAL FOR RESEARCH
IN APPLIED SCIENCE & ENGINEERING TECHNOLOGY

Volume: 14 Issue: I Month of publication: January 2026

DOI: https://doi.org/10.22214/ijraset.2026.77146

www.ijraset.com
Call: 08813907089 | E-mail ID: ijraset@gmail.com

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue I Jan 2026- Available at www.ijraset.com

Web Application for Secure Transmission of Data

Rama Subbarao Mamidipalli[1], Gangadhar Rao Mallavarapu[2], Rakesh Surya Gangadhar Mangam[3], Satya Siva Rama Krishna Bonthu[4], Rohith Sai Ramareddy Bethireddy[5], Ganga Bhavani B[6]

[1, 2, 3, 4, 5]*Department of Computer Science and Engineering, Bonam Venkata Chalamayya Engineering College, Affiliated to JNTU Kakinada, Andhra Pradesh-533210, India*

[6]*Project Guide, Department of Computer Science and Engineering, Bonam Venkata Chalamayya Engineering College, Affiliated to JNTU Kakinada, Andhra Pradesh-533210, India*

***Abstract:** In the modern digital era, the secure transmission of data over the internet in the digital world is highly required due to the rising growth in cybercrime incidents. Upon completion of the research work in the digital domain, it will be possible to develop an entire web-based application based on the concept of secure data transmission by using a combination of different hybrid techniques of cryptography based on AES, along with the RSA key exchange algorithm. With proper implementation, the strengths of both the encryption standards could be utilized while the weaknesses of the respective standards could be avoided. The application is developed with HTML and CSS for the frontend, an intuitive interface, while Python's Flask framework and MySQL are used as the server side for robust backend processing. This creates a smooth experience for the users in encrypting and also decrypting the information in a secure manner. Here, the Fernet symmetric authenticated encryption scheme from the Python Cryptography Library has been used for effective deployment of the algorithms within the given architecture. Further, the performance evaluation of the proposed model ensures the achievement of the best standards in terms of security and the level of processing efficiency, making it highly suitable for the practical implementation of secure communication within the domain. Thus, the proposed model can solve the challenges of efficiency alongside the security challenges that occur within the environment of secure communications.*

***Keywords:** AES algorithm, RSA algorithm, Cryptography, Data Security, Secure Communication*

I. INTRODUCTION

With an exponential growth in the usage and demand of internet services and different modes of internet communication across the world, issues of data security and privacy have cropped up as two significant issues. Today, when many vital information exchanges take place through different media of communications, issues of information confidentiality, integrity, and authenticity have become extremely significant issues not just in terms of businesses and governments but individuals across the world[1]. The world of digits and data security is reaching unprecedented threat vectors of unauthorized activities like unauthorized access, mass data breach attacks, and many more. Moreover, statistics show that this cost will amount to trillions of dollars in a single year, while millions of records will be at a greater risk of exposure because of a breach in this kind of data. The alarming situation of cybercrimes, therefore, proves that secure encryption methods will require more attention to secure digital information. Apparently, traditional security balances cannot fight cyber threats effectively, thus requiring more sophisticated encryption methods. "The fundamental backbone for information security is based on cryptography, which provides various tools to safeguard information from access and misuse through the use of mathematical algorithms during transmission and storage" [2]. Amongst various tools and methods used in cryptography for information security, the encryption algorithm is the most basic and plays a very significant role in converting readable information into an unreadable format for preventing unauthorized accesses of information while in transit. Advanced Encryption Algorithm: Advanced Encryption Algorithm, also known as Advanced Encryption Standard, is considered one of the most secure encryption techniques that has less computational complexity in its application to avail maximum encryption benefits. Presently, it is being used by governments across the world as a standard encryption mechanism to store data securely in financial institutions, technological businesses, and the government itself to keep their private data. Nevertheless, there are also a few limitations that have been associated with the symmetric cryptogram encrypting component. To be a little more precise, one generalized aspect considered here is related to the safe receipt of the encrypting key such that it is not intercepted by any other "evil entity." If the key involved gets compromised in the "key exchange process" itself, the whole idea of security fails. As far as the implementation of the "RSA algorithm of safe key exchange is considered, it is clearly seen that the implementation of this algorithm is fully compliant with the "asymmetric key cryptography principle."

©IJRASET: All Rights are Reserved | SJ Impact Factor 7.538 | ISRA Journal Impact Factor 7.894 |

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue I Jan 2026- Available at www.ijraset.com

As opposed to the requirement of "key exchange" involved in the implementation of any "symmetric encrypting option," this option nullifies this issue of "key exchange" in a very elegant way without involving any "key sharing" between the two organizations. To outline the main aim of this study, the aim of this particular paper aims at designing an integrated web-based application through the merging of the AES and RSA algorithms in an attempt to create an even more robust algorithm for the sake of designing an even more robust encryption technique that will be capable of efficaciously leveraging the best from the combining algorithms in the designing of this particular paper. This particular paper aims to address the motivation behind the need for an important user interface through the use of an efficient encryption technique.

II. LITERATURE REVIEW

Existing studies based on secure data transmission techniques have widely evaluated several types of cryptography techniques with various advantages and disadvantages. In symmetric cryptography techniques like DES, Triple DES, or AES, data encryption techniques are widely evaluated with various advantages. The security of the data is efficiently achieved as the large amount of data can be easily encrypted with less processing time. Although the DES is a very crucial technique with historical values, it is no longer deemed a secure technique since the length of the key is too low to withstand brute force attacks. TripleDES has improved security by applying the DES algorithm three times, with the disadvantage of the processing time. AES evolved as an improved replacement for DES, resulting from a detailed selection process implemented and conducted by the National Institute of Standards and Technology (NIST). Its adoption as a federal standard represented a major leap forward in symmetric crypto technology. However, the key problem in utilizing symmetric crypto systems has remained the distribution of keys, i.e., without the interception of the encryption key during transmission. The elegance of key exchange lies in asymmetric encryption algorithms, for example, RSA; however, this is computationally expensive in handling large datasets of information. Prashant et al., in their study (2024), presented a comprehensive comparative study of AES and RSA algorithms, along with other cryptographic techniques, indicating that while AES algorithms are efficient in terms of data processing speeds, in comparison, they are 1000 times faster compared to RSA in handling larger datasets, while RSA algorithms are better in handling key transfer due to public key infrastructures, as demonstrated in Prashant et al. (2024).

Sood and Kaur undertook an extensive study of various encryption techniques like RSA, DES, AES, etc. The authors found that out of all the data encryption techniques analysed based on security strength, key management techniques, and complexity of computation involved in each method, AES has the best compromise between security and efficiency of operation in the modern world of increasing data needs and threats to data security and integrity. Singh and Supriya carried out an analysis of various stages in the evolution of encryption algorithms, beginning with historical forms of encryption up to recent computational methods, thus substantiating their claim of AES being the industry standard algorithm because of its resistance to decryption, ease of implementation in both software and hardware forms, and its versatile application in various forms [3]. The authors' findings highlighted how better security solutions are developed by a combination of both methods than either being applied in isolation to compensate for their inherent deficiencies.

Kumari and Mahato (2025) discussed the "fascinating evolution" that can be traced regarding the history of cryptography that starts from the substitution ciphers used by Julius Caesar until the present day "quantum-resistant" form of encryption that can help handle the "inherent need for increasingly robust forms of encryption that grow exponentially alongside computing power." They identified that the "hybrid forms" that depend on more than "a single form of encryption provide far more robust security for modern applications by creating an exponentially harder problem." The literature provided within the Globus Toolkit documentation offers a level of insight pertaining to concepts revolving around security and encryption in a distributed system, as well as the importance of secure protocols within a networked scenario [5].

It highlights that a strong level of encryption must exist within a distributed system. Though there have been extensive theoretical explorations of each of these algorithms, there has been a huge gap identified in their overall practical implementation that incorporates AES as well as RSA in an easy and interactive manner available for end users through internet applications or platforms. Moreover, most of such applications that are available are highly centered around their theoretical and practical analysis in controlled environments rather than their implementation in easier manners for users to utilize in real-time applications or scenarios. The significance of doing this research is that it is aimed to serve as a reliable platform that incorporates ease of use in association with high levels of security requirements during its overall implementation process.

©IJRASET: All Rights are Reserved | SJ Impact Factor 7.538 | ISRA Journal Impact Factor 7.894 |

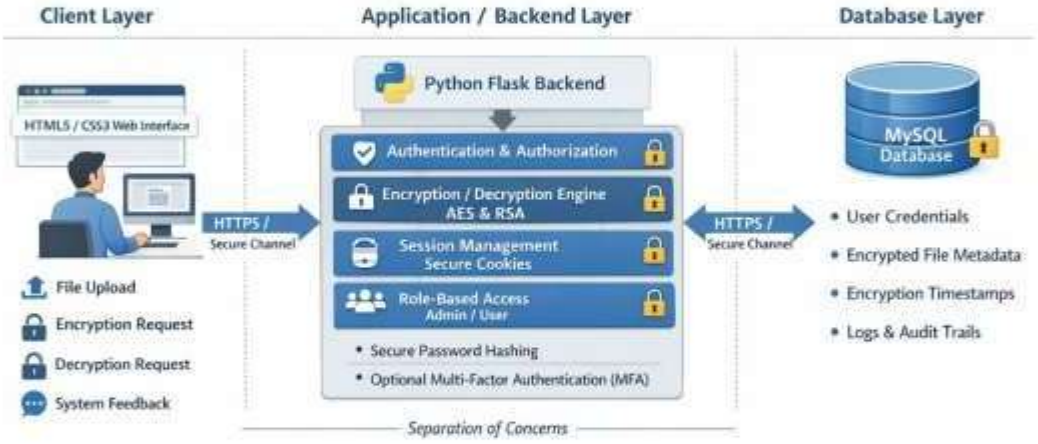
International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue I Jan 2026- Available at www.ijraset.com

III. METHODS

A. System Architecture

The system that is proposed has a highly structured client-server architecture. Users will interact with this system through a normal web interface. They will upload files to perform corresponding encryption/decryption operations. This application is developed with front-end tools such as HTML5 and CSS3. Normal web development principles have been used to make this application highly user-friendly. Users will have a good view of system feedback during encryption/decryption operations. The backend layer employs the concept of lightweight and powerful web development technologies like Python Flask. It handles encryption operations and deals efficiently with user authentication and database interactions. It ensures clean separation of concerns. It has built-in security features.

MySQL database management will be utilized, storing the users' credentials, encrypted file information, information about the encrypted files, timestamps relating to the encryption process, and log information. The design scheme will enable maintainability, as well as optimize query performance to enable the retrieval of files quickly. User authentication techniques are used to ensure that the users who are accessing the encryption and decryption services are valid. The system utilizes secure password hashing with widely accepted standards and protocols for session handling, with the use of secure cookies for the purpose of retaining the state information of users. Role-based security is used for the administration functions.



B. AES Algorithm Working

AES is a symmetric block cipher that uses a block size of 128 bits, along with a key size of 128, 192, or 256 bits, to encrypt blocks of information, where the larger the key size, the more secure the information will be [1]. Here, a 256-bit key size has been used to ensure maximum security for the information stored in the database, as it can safely counter any present or future kinds of cryptanalysis attacks performed on the information. The process of AES encryption takes place through several rounds of transformation, with 10 rounds for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds of transformation for 256-bit keys. The round consists of four different operations:

1) *SubBytes:* Each byte of the state array in the Rijndael algorithm is replaced by another byte from a pre-defined substitution box. Adding this non-linearity makes it difficult to apply differential and linear cryptanalysis.
2) *ShiftRows:* The rows of the state matrix are shifted in a cyclical way and by different displacements, where row 0 stays unchanged, row 1 shifts left by 1 byte, row 2 by 2 bytes, and row 3 by 3 bytes. This increases dispersion across the algorithm, with a modification in a certain input having also changed several bytes in the output.
3) *MixColumns:* This operation mixes the columns using multiplication of matrices over the Galois Field GF(2^8), which results in more diffusion of data in the block, so each input bit affects many output bits. The MixColumns step is excluded from the last round.
4) *AddRoundKey:* By using XOR operations, the state is combined with one round key, which is derived from the main encryption key. This introduces the key material into each round.

©IJRASET: All Rights are Reserved | SJ Impact Factor 7.538 | ISRA Journal Impact Factor 7.894 |

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue I Jan 2026- Available at www.ijraset.com

The process of decryption is the inverse of the encryption process. The inverse steps involve InvSubBytes, InvShiftRows, InvMixColumns, and AddRoundKey in the opposite order. The merits of AES include faster encryption rates, better security, measured by resistance to all types of attacks, including differential cryptanalysis and linear cryptanalysis, along with easy implementation in hardware as well as software [2]. The modern processors have been equipped with the AES instruction sets that improve the rate of the encryption and decryption process.

C. RSA Algorithm Working

RSA is a public-key or "asymmetric-key" encryption method that relies upon a pair of mathematically linked keys: a public encryption key may be openly distributed, whereas a private key for decryption must be kept secret [3]. The basis upon which RSA operates is the difficulty in processing the reversal of a product of large prime numbers, a problem for which there is as yet no efficient method or algorithm implemented anywhere in the world in computer systems [1]. Some of the most critical steps in the key generation process are considered to be the selection of two large prime numbers that are often 1024 or 2048 bits long each, the computation of the product of the two prime values as the modulus value 'n', the computation of the value represented as $\varphi(n) = (p - 1)(q - 1)$, the selection of the public exponent 'e', which is necessarily higher than 65537 for efficiency considerations and is coprime with $\varphi(n)$, and the computation of the private exponent 'd', which is needed to satisfy the equation $(d * e) \bmod \varphi(n) = 1$.
During the encryption process, the text message sent is converted into integers. The integer form of the message is then raised to some power e modulo n: C = M^e mod n. While in the process of decrypting the data, the ciphertext that has been sent receives the value of the modular inverse of the original integer raised to some power d modulo n: M = C^d mod n. The basic formula that has been used shows that only the recipient with the matching private key will be able to decrypt any data that has been encrypted with their matching public key [4]. In this case, the RSA algorithm has an important role in key exchange security rather than data encryption. In fact, the key used for encryption by the AES algorithm has to be encrypted using the RSA key of the receiver before the information is actually sent over the channel. In this way, only the authorized party having the corresponding private key can access the actual key for decryption using the AES algorithm.

IV. RESULTS

The experimental environment was a local environment, in which a system that has 8 GB of memory, an Intel i5 processor running at 2.4 GHz, Windows11 as its operating system, along with Python 3.8 and Flask 2.0. The system tested various cases of system performance by testing files of various sizes, from 1KB of plain text up to 10 megabytes of multimedia content. Samples provided indicated that a full encryption/decryption cycle with perfect integrity was achievable. The 1 MB text file took 0.15 seconds to encrypt with AES 256. The RSA file encryption of the 256-bit AES took 0.03 seconds regardless of file size. In terms of decrypt times, this is close to the encrypt times. RSA took 0.04 seconds to decrypt, while AES took 0.15 seconds to decrypt this file. Further, the efficiency of the code is demonstrated by the performance evaluation result that the time consumed for file encryption is directly proportional to the size of the file. The code maintains consistency in efficiency with various types of files, such as text documents, images, video clips, or compressed files. When the size of the file is considered as 5MB for the pdf file encryption, the code takes a total of 0.68 seconds. When the size of the video file is considered as 10MB, the time consumed is 1.3 seconds. Further, the time consumed for the encryption of the key is constant for any file size or any file type and is considered to be 0.03 seconds. The file size, as opposed to processing time analysis, demonstrated that it is efficient to handle files of up to 10MB without any noticeable deterioration in performance or any signs of excessive use of memory. The method was seen to be far more efficient and effective as opposed to relying on using RSA for large files alone, which would have seen its processing time 50-100 times higher and also been limited owing to its maximum message size constraint. Security effectiveness testing has assured us that the encrypted files are completely unreadable without the correct decryption keys. It has been ensured that if incorrect keys are used while attempting to decrypt the files, total failure with proper error messages is achieved. Simulations of brute-force attacks have assured us that attempting such an approach to break the encryption is computationally impossible in any timely fashion. The system has been successful in preventing unauthorized access, data tampering, and replay attacks through the proper implementation of security best practices.

V. DISCUSSION

Results comprehensively demonstrate how the hybrid model of combining AES and RSA outperforms and sustains an effective, practical solution for safe data transmission in web environments. The hybrid approach has been strategically using AES's computational efficiency for encrypting large data volumes by utilizing RSA for its secure key exchange method to address the very symptom-a symmetric key distribution challenge-that has classically hindered the widespread cryptographic deployment.

©IJRASET: All Rights are Reserved | SJ Impact Factor 7.538 | ISRA Journal Impact Factor 7.894 |

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue I Jan 2026- Available at www.ijraset.com

The implementation of the proposed system would therefore offer an optimum balance between the twin requirements of security and performance, compared to existing encryption-based systems that use either symmetric or asymmetric methods exclusively. Systems based on RSA alone for data encryption suffer from severe performance bottlenecks when large files are involved and become practically unusable beyond a few kilobytes because of computational complexity issues. Systems based purely on AES, on the other hand, face problems of secure key distribution, which invariably forces organizations to adopt out-of-band key exchange methods. Accordingly, the advantages of employing both AES as well as RSA are numerous; in line with this fact, the encryption strength provided by 256-bit encryption in AES would effectively act as a barrier to brute-force hacking attempts, the implications of key distribution problems would be effectively handled by employing secure key transmission in RSA encryption, the performance characteristics of this system would facilitate real-time encryption of files with moderate sizes, as well as demonstrate defense in depth because of the layered encryption employed. This system would effectively provide high levels of security for data, including financial data, healthcare data, and confidential business data. There are naturally security/performance compromises to be made here. While the security of the key is very high with an RSA encryption, the computational burden is very low compared to a regular AES encryption. The system attempts to manage this in two ways: the computational burden will not be an issue whatsoever, as the RSA is used only for key encryption rather than data encryption. Naturally, there is a minor increase in memory usage because two contexts are needed: one for AES and another for RSA.

The real-time applicability of the system extends to a variety of practical scenarios, which include secure file sharing within an organization, confidential document transmission between business partners, protected cloud storage in which even the service provider does not have any access to unencrypted data, secure email attachments, healthcare data exchange as per privacy regulations, and financial transaction security. The web-based interface makes advanced cryptography accessible to users without requiring deep technical knowledge, thus promoting wider adoption of security best practices. Some future enhancements to this may include the use of digital signatures for authentication and non-repudiation of messages, inclusion of more encryption algorithms to enable algorithm agility, and integration with enterprise identity management systems. Others include key escrow mechanisms that ensure organizational data recovery, file compression before encryption to minimize costs in storage and transmission, and finally, the development of mobile applications to ensure cross-operability.

VI. CONCLUSION

Overall, the study has effectively created and evaluated a comprehensive internet application that is able to facilitate secure data transmission via a technique employing the best features of both symmetric and asymmetric data encryption. Implementation of the AES technique has proved highly successful for the efficient encryption of data via the utilization of a technique employing the features of both symmetric and asymmetric data encryption. It can be seen that the proposed system fulfils its proposed objectives with a facility that can perform user-friendly and understandable file encryption as well as decryption, even for users who are not highly proficient in matters of information security. The proposed approach also passed performance evaluations that showed that not only does it perform optimally, but it also ensures that there are strong security standards, enough to counter current cyber threats, with the two concepts covering each other well, as one method does not have what the other lacks. It successfully implemented hybrid encryption using current, open standards for encryption algorithms. The intuitive web interface is easily accessible from any modern browser. It has very efficient processing times for files up to 10 MB, with linear scalability. Key management mechanisms are secure, ensuring protection for the cryptographic keys throughout their life cycle. Authentication and authorization controls are strong, preventing unauthorized access. This system demonstrates how sophisticated cryptographic security can be made available without compromising rigor in security. Future enhancements may be the support for large files by the use of streaming encryption, the use of digital signatures for authentication and non-repudiation, more encryption algorithms to enable algorithm agility, the development of mobile applications for a wider platform support, and the implementation of advanced key management features, including key rotation and escrow mechanisms in enterprise deployments. With this, the web application demonstrates that hybrid cryptography systems show practical and deployable solutions to provide security in modern digital communications; thus, a balance between the two important and competing requirements of security strength and computational efficiency can be well achieved. The research applied therein proves that any efficient and sound cryptographic implementation could be robust for protection while remaining practical for everyday usage.

REFERENCES

[1] Prashant, Md Sohail Haque, Amrinder Kaur, Pankaj Yadav. (2024). Comparative Analysis of AES and RSA with Other Encryption Techniques for Secure Communication. International Journal of Scientific Research in Computer Science, Engineering and Information Technology.

©IJRASET: All Rights are Reserved | SJ Impact Factor 7.538 | ISRA Journal Impact Factor 7.894 |

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue I Jan 2026- Available at www.ijraset.com

[2] Sood, Kaur, S. A Literature Review on RSA, DES, and AES Encryption Algorithms. SCRS Publications.
[3] Singh, Supriya. A Study of Encryption Algorithms (RSA, DES, 3DES, and AES) for Information Security. International Journal of Computer Applications.
[4] Kumari, Mahato, T. K. (2025). The Evolution of Secure Communication: Analysing Cryptographic Methods from Ancient to Modern Era. International Research Journal.
[5] Globus Toolkit Documentation. Security and Encryption Concepts in Distributed Systems.

©IJRASET: All Rights are Reserved | SJ Impact Factor 7.538 | ISRA Journal Impact Factor 7.894 |

doi
cross ref
10.22214/IJRASET
INDEX COPERNICUS
45.98
JISRAF
IMPACT FACTOR: 7.129
TOGETHER WE REACH THE GOAL
IMPACT FACTOR: 7.429
INTERNATIONAL JOURNAL FOR RESEARCH
IN APPLIED SCIENCE & ENGINEERING TECHNOLOGY
Call : 08813907089 (24*7 Support on Whatsapp)

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: 7.429

45.98 INDEX COPERNICUS

THOMSON REUTERS

doi cross ref

TOGETHER WE REACH THE GOAL

Certificate

It is here by certified that the paper ID : IJRASET77146, entitled

Web Application for Secure Transmission of Data

by

Rama Subbarao Mamidipalli

after review is found suitable and has been published in Volume 14, Issue I, January 2026

in

International Journal for Research in Applied Science & Engineering Technology

(International Peer Reviewed and Refereed Journal)

Good luck for your future endeavors

Editor in Chief, IJRASET

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: 7.429

45.98 INDEX COPERNICUS

THOMSON REUTERS

doi cross ref

TOGETHER WE REACH THE GOAL

Certificate

It is here by certified that the paper ID : IJRASET77146, entitled

Web Application for Secure Transmission of Data

by

Gangadhar Rao Mallavarapu

after review is found suitable and has been published in Volume 14, Issue I, January 2026

in

International Journal for Research in Applied Science & Engineering Technology

(International Peer Reviewed and Refereed Journal)

Good luck for your future endeavors

Editor in Chief, IJRASET

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: 7.429

45.98 INDEX COPERNICUS

THOMSON REUTERS

doi cross ref

TOGETHER WE REACH THE GOAL

Certificate

It is here by certified that the paper ID : IJRASET77146, entitled

Web Application for Secure Transmission of Data

by

Rakesh Surya Gangadhar Mangam

after review is found suitable and has been published in Volume 14, Issue I, January 2026

in

International Journal for Research in Applied Science & Engineering Technology

(International Peer Reviewed and Refereed Journal)

Good luck for your future endeavors

Editor in Chief, iJRASET

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: 7.429

45.98 INDEX COPERNICUS

THOMSON REUTERS

doi cross ref

TOGETHER WE REACH THE GOAL

Certificate

It is here by certified that the paper ID : IJRASET77146, entitled

Web Application for Secure Transmission of Data

by

Satya Siva Rama Krishna Bonthu

after review is found suitable and has been published in Volume 14, Issue I, January 2026

in

International Journal for Research in Applied Science & Engineering Technology

(International Peer Reviewed and Refereed Journal)

Good luck for your future endeavors

Editor in Chief, iJRASET

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: 7.429

45.98 INDEX COPERNICUS

THOMSON REUTERS

doi cross ref

TOGETHER WE REACH THE GOAL

Certificate

It is here by certified that the paper ID : IJRASET77146, entitled

Web Application for Secure Transmission of Data

by

Rohith Sai Ramareddy Bethireddy

after review is found suitable and has been published in Volume 14, Issue I, January 2026

in

International Journal for Research in Applied Science & Engineering Technology

(International Peer Reviewed and Refereed Journal)

Good luck for your future endeavors

Editor in Chief, iJRASET

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: 7.429

45.98 INDEX COPERNICUS

THOMSON REUTERS

doi cross ref

TOGETHER WE REACH THE GOAL

Certificate

It is here by certified that the paper ID : IJRASET77146, entitled

Web Application for Secure Transmission of Data

by

Ganga Bhavani B

after review is found suitable and has been published in Volume 14, Issue I, January 2026

in

International Journal for Research in Applied Science & Engineering Technology

(International Peer Reviewed and Refereed Journal)

Good luck for your future endeavors

Editor in Chief, iJRASET